

EN3160 - Image Processing Assignment 1

Name: Hasith Nettikumara

Index Number: 230436X

This notebook contains the implementations and results for Assignment 1 on intensity transformations and neighborhood filtering.

```
In [ ]: %pip install matplotlib opencv-python numpy
```

```
In [2]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

```
In [3]: # Folder containing the assignment images
IMAGE_DIR = Path("images")

# Check that the folder exists
print("Image directory exists:", IMAGE_DIR.exists())
```

Image directory exists: True

```
In [4]: def show_image(image, title="", cmap=None, figsize=(6, 6)):
plt.figure(figsize=figsize)

if image.ndim == 3:
    # OpenCV loads color images in BGR format
    image_rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
    plt.imshow(image_rgb)
else:
    plt.imshow(image, cmap=cmap if cmap else "gray")

plt.title(title)
plt.axis("off")
plt.show()
```

Question 1 - Piecewise Linear Intensity Transformation

A piecewise linear intensity transformation is applied to the given grayscale image using a set of specified breakpoints. Linear interpolation is used to determine the output intensity values between consecutive breakpoints.

```
In [ ]: print("Available assignment images:")
```

```
for q1_available_image_path in IMAGE_DIR.iterdir():  
    print(q1_available_image_path.name)
```

```
In [145... q1_portrait_file = IMAGE_DIR / "q1.jpeg"  
  
q1_portrait_gray = cv.imread(  
    str(q1_portrait_file),  
    cv.IMREAD_GRAYSCALE  
)  
  
if q1_portrait_gray is None:  
    raise FileNotFoundError(f"Could not load Question 1 image: {q1_portrait_file}")  
  
print("Image shape:", q1_portrait_gray.shape)  
print("Image datatype:", q1_portrait_gray.dtype)  
print("Minimum intensity:", q1_portrait_gray.min())  
print("Maximum intensity:", q1_portrait_gray.max())
```

Image shape: (810, 720)

Image datatype: uint8

Minimum intensity: 0

Maximum intensity: 255

```
In [147... plt.figure(figsize=(6, 6))  
  
plt.imshow(q1_portrait_gray, cmap="gray", vmin=0, vmax=255)  
  
plt.title("Original Image - Question 1")  
plt.axis("off")  
plt.show()
```

Original Image - Question 1



```
In [ ]: q1_breakpoints_reference = np.array([[0, 0],[50, 50],[100, 150],[150, 255],[255,
q1_breakpoints_trial_a = np.array([[0, 0],[40, 30],[90, 110],[150, 210],[210, 2
q1_breakpoints_trial_b = np.array([[0, 0],[50, 25],[100, 125],[160, 220],[220, 2
q1_breakpoints_trial_c = np.array([[0, 0],[40, 30],[90, 100],[150, 175],[210, 2
q1_breakpoints_trial_d = np.array([[0, 0],[40, 28],[90, 98],[150, 168],[210, 22
q1_breakpoints_trial_e = np.array([[0, 0],[40, 32],[90, 102],[150, 170],[210, 2
```

```
In [149... def intensity_transform(im, breakpoints):
    input_levels = breakpoints[:, 0]
    output_levels = breakpoints[:, 1]

    transformed_image = np.interp(im.astype(np.float32),input_levels,output_levels)
    transformed_image = np.clip(transformed_image,0,255)

    return transformed_image.astype(np.uint8)
```

```
In [150... q1_output_reference = intensity_transform(q1_portrait_gray,q1_breakpoints_reference)
q1_output_trial_a = intensity_transform(q1_portrait_gray,q1_breakpoints_trial_a)
q1_output_trial_b = intensity_transform(q1_portrait_gray,q1_breakpoints_trial_b)
q1_output_trial_c = intensity_transform(q1_portrait_gray,q1_breakpoints_trial_c)
q1_output_trial_d = intensity_transform(q1_portrait_gray,q1_breakpoints_trial_d)
q1_output_trial_e = intensity_transform(q1_portrait_gray,q1_breakpoints_trial_e)
```

```
In [ ]: q1_trial_figure, q1_trial_axes = plt.subplots(2,4,figsize=(16, 10))

q1_trial_images = [q1_portrait_gray,q1_output_reference,q1_output_trial_a,q1_output_trial_b]

q1_trial_titles = ["Original","Reference","Trial A","Trial B","Trial C","Trial D","Trial E","Trial F"]

# Flatten the 2x4 axes array so we can index it easily
q1_trial_axes_flat = q1_trial_axes.flatten()

for q1_panel_index in range(len(q1_trial_images)):

    q1_trial_axes_flat[q1_panel_index].imshow(q1_trial_images[q1_panel_index],cmap='gray')

    q1_trial_axes_flat[q1_panel_index].set_title(q1_trial_titles[q1_panel_index])

    q1_trial_axes_flat[q1_panel_index].axis("off")

# Hide the unused 8th subplot
q1_trial_axes_flat[-1].axis("off")

plt.tight_layout()
plt.show()
```



```
In [ ]: # Select Trial D as the final intensity transformation
q1_selected_breakpoints = q1_breakpoints_trial_d.copy()

q1_selected_output = intensity_transform(q1_portrait_gray,q1_selected_breakpoints)
```

```
In [ ]: q1_final_figure, q1_final_axes = plt.subplots(1,2,figsize=(12, 5))

# Original image
q1_final_axes[0].imshow(q1_portrait_gray,cmap="gray",vmin=0,vmax=255)
```

```

q1_final_axes[0].set_title("Original Image")
q1_final_axes[0].axis("off")

# Selected transformed image - Trial D
q1_final_axes[1].imshow(q1_selected_output, cmap="gray", vmin=0, vmax=255)

q1_final_axes[1].set_title("Selected Transformed Image (Trial D)")
q1_final_axes[1].axis("off")

plt.tight_layout()
plt.show()

```

Original Image



Selected Transformed Image (Trial D)



```

In [ ]: q1_selected_intensity_axis = np.arange(256)

q1_selected_mapping = np.interp(q1_selected_intensity_axis, q1_selected_breakpoints[
plt.figure(figsize=(7, 6))

plt.plot(q1_selected_intensity_axis, q1_selected_mapping, linewidth=2)

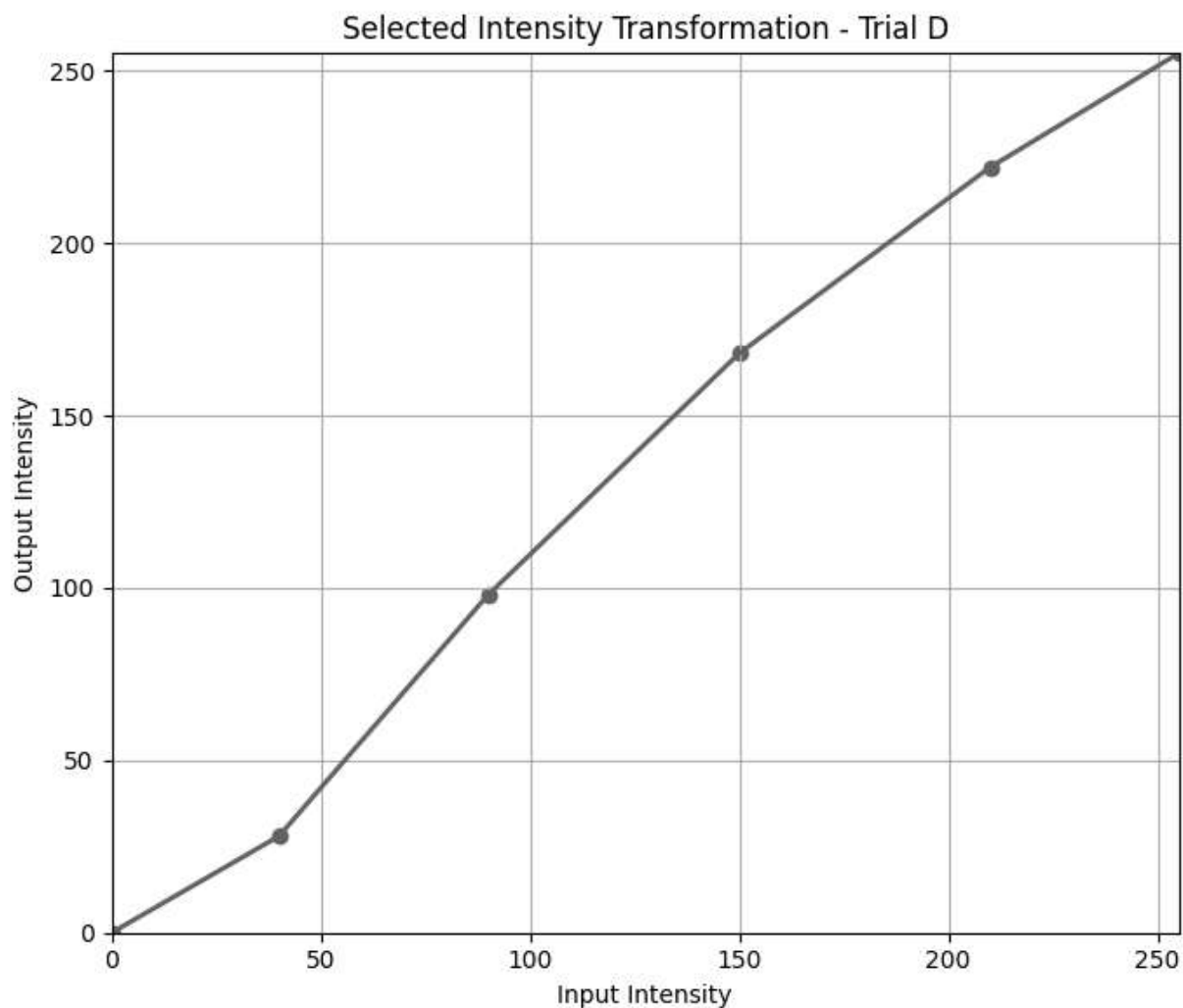
plt.scatter(q1_selected_breakpoints[:, 0], q1_selected_breakpoints[:, 1])

plt.xlabel("Input Intensity")
plt.ylabel("Output Intensity")
plt.title("Selected Intensity Transformation - Trial D")

plt.xlim(0, 255)
plt.ylim(0, 255)
plt.grid(True)

plt.tight_layout()
plt.show()

```



Question 2 - White Matter and Gray Matter Enhancement

Piecewise linear intensity transformations are used to separately accentuate white matter and gray matter in the proton-density brain image.

The image intensity distribution is first examined to identify suitable intensity ranges for the two tissue types. Separate transformations are then designed for white matter and gray matter enhancement.

```
In [ ]: q2_brain_file = IMAGE_DIR / "q2.jpeg"

q2_brain_gray = cv.imread(str(q2_brain_file),cv.IMREAD_GRAYSCALE)

if q2_brain_gray is None:
    raise FileNotFoundError(f"Could not load Question 2 image: {q2_brain_file}")

print("Brain image shape:", q2_brain_gray.shape)
print("Brain image datatype:", q2_brain_gray.dtype)
```

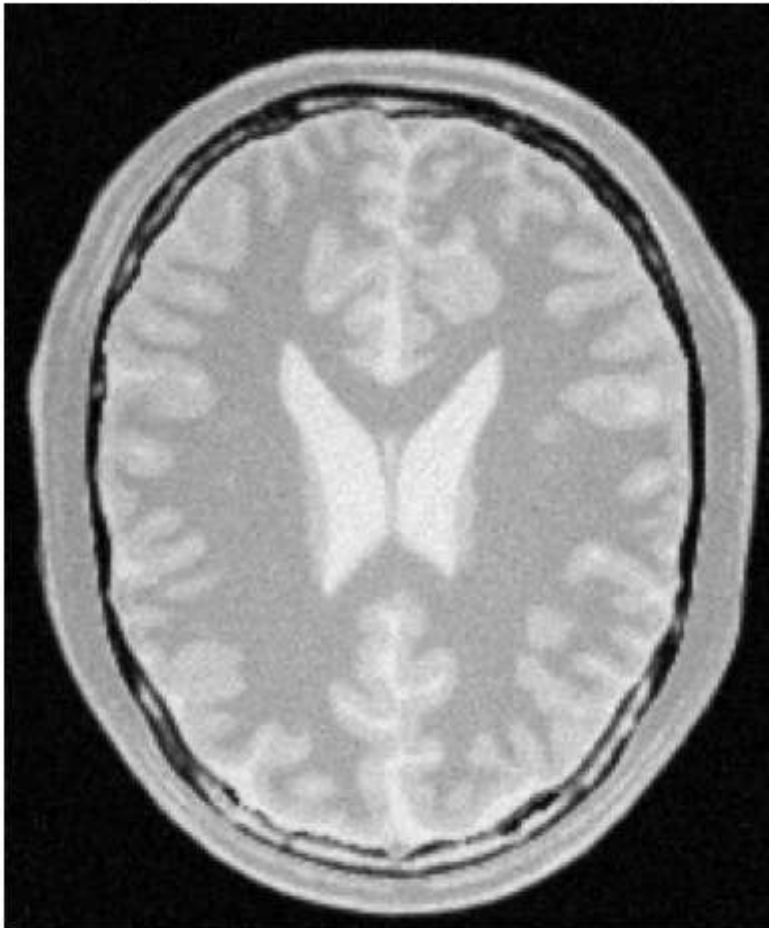
```
print("Minimum intensity:", q2_brain_gray.min())
print("Maximum intensity:", q2_brain_gray.max())
```

```
Brain image shape: (217, 181)
Brain image datatype: uint8
Minimum intensity: 0
Maximum intensity: 251
```

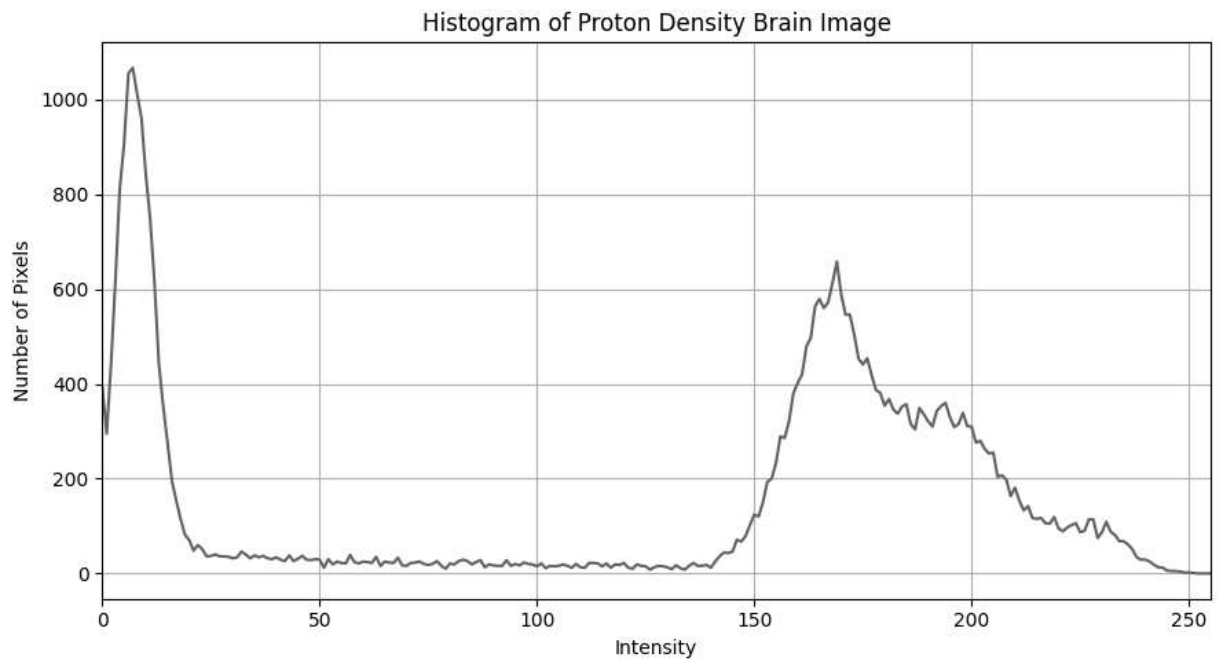
```
In [ ]: plt.figure(figsize=(6, 6))
plt.imshow(q2_brain_gray,cmap="gray",vmin=0,vmax=255)
plt.title("Original Proton Density Brain Image")
plt.axis("off")

plt.show()
```

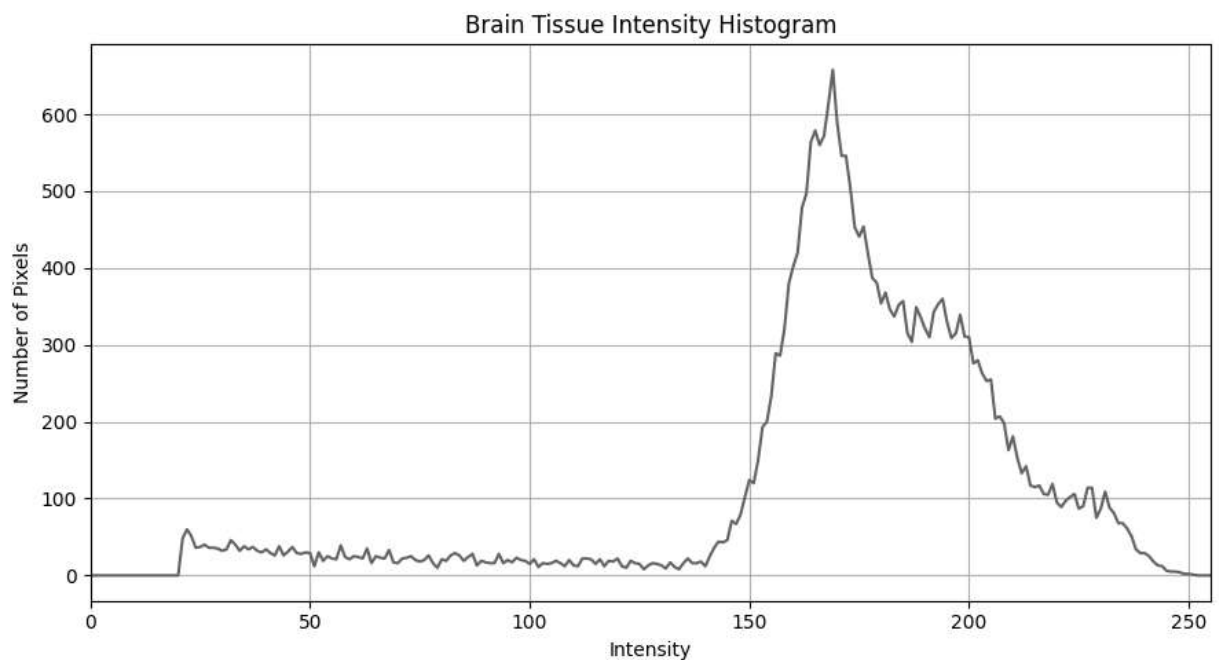
Original Proton Density Brain Image



```
In [152... q2_histogram_counts,q2_histogram_bins = np.histogram(q2_brain_gray.flatten(),bins=256)
plt.figure(figsize=(9,5))
plt.plot(np.arange(256),q2_histogram_counts)
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.title("Histogram of Proton Density Brain Image")
plt.xlim(0,255)
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [ ]: q2_tissue_pixels = q2_brain_gray[q2_brain_gray > 20]
q2_tissue_histogram,_ = np.histogram(q2_tissue_pixels,bins=256,range=(0,256))
plt.figure(figsize=(9,5))
plt.plot(np.arange(256),q2_tissue_histogram)
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.title("Brain Tissue Intensity Histogram")
plt.xlim(0,255)
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [ ]: q2_percentile_levels = [5,10,25,50,75,90,95]
q2_tissue_percentiles = np.percentile(q2_tissue_pixels,q2_percentile_levels)
```

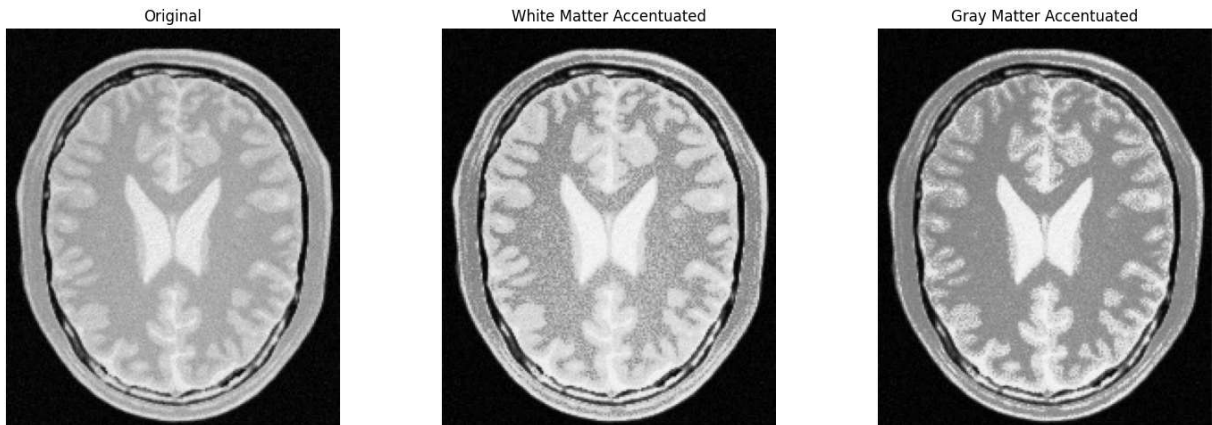
```
for q2_percentile_label,q2_percentile_value in zip(q2_percentile_levels,q2_tissue_p
    print(f"{q2_percentile_label:>2}th percentile: {q2_percentile_value:.2f}")
```

```
5th percentile: 64.00
10th percentile: 141.00
25th percentile: 163.00
50th percentile: 175.00
75th percentile: 195.00
90th percentile: 211.00
95th percentile: 223.00
```

```
In [153... # Final piecewise-linear transformation for white matter enhancement
q2_white_matter_breakpoints = np.array([[0,0],[120,105],[145,125],[160,145],[180,20
# Final piecewise-linear transformation for gray matter enhancement
q2_gray_matter_breakpoints = np.array([[0,0],[135,115],[165,140],[185,165],[205,220
```

```
In [154... q2_white_matter_output = intensity_transform(q2_brain_gray,q2_white_matter_breakpoi
q2_gray_matter_output = intensity_transform(q2_brain_gray,q2_gray_matter_breakpoint
```

```
In [155... q2_final_figure,q2_final_axes = plt.subplots(1,3,figsize=(15,5))
q2_final_images = [q2_brain_gray,q2_white_matter_output,q2_gray_matter_output]
q2_final_titles = ["Original","White Matter Accentuated","Gray Matter Accentuated"]
for q2_final_index in range(3):
    q2_final_axes[q2_final_index].imshow(q2_final_images[q2_final_index],cmap="gray
    q2_final_axes[q2_final_index].set_title(q2_final_titles[q2_final_index])
    q2_final_axes[q2_final_index].axis("off")
plt.tight_layout()
plt.show()
```



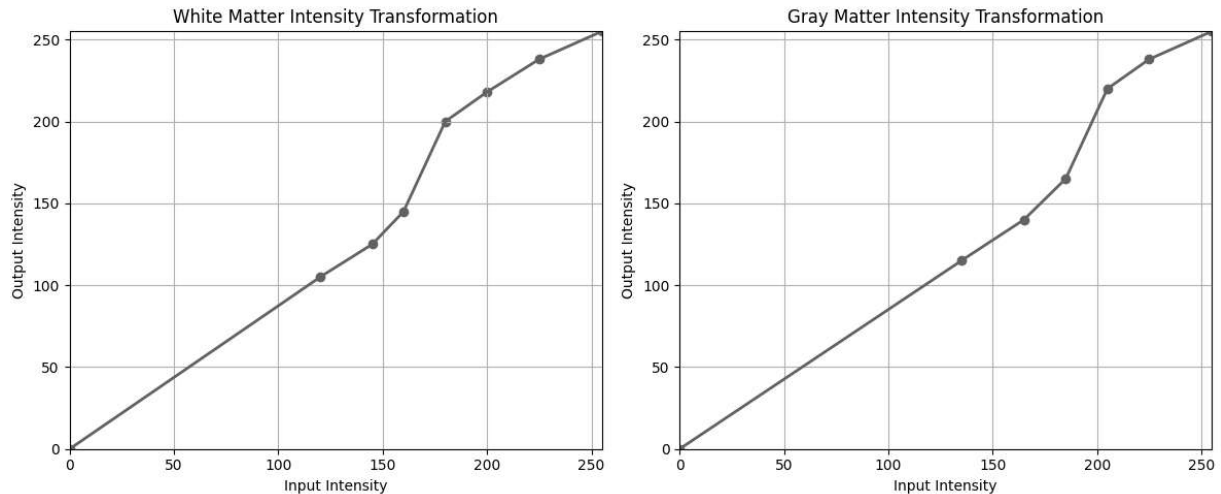
```
In [156... q2_intensity_axis = np.arange(256)
q2_white_matter_curve = np.interp(q2_intensity_axis,q2_white_matter_breakpoints[:,0
q2_gray_matter_curve = np.interp(q2_intensity_axis,q2_gray_matter_breakpoints[:,0],
```

```
In [ ]: q2_transform_figure,q2_transform_axes = plt.subplots(1,2,figsize=(12,5))
# White matter transformation
q2_transform_axes[0].plot(q2_intensity_axis,q2_white_matter_curve,linewidth=2)
q2_transform_axes[0].scatter(q2_white_matter_breakpoints[:,0],q2_white_matter_break
q2_transform_axes[0].set_title("White Matter Intensity Transformation")
q2_transform_axes[0].set_xlabel("Input Intensity")
q2_transform_axes[0].set_ylabel("Output Intensity")
q2_transform_axes[0].set_xlim(0,255)
q2_transform_axes[0].set_ylim(0,255)
```

```

q2_transform_axes[0].grid(True)
# Gray matter transformation
q2_transform_axes[1].plot(q2_intensity_axis,q2_gray_matter_curve,linewidth=2)
q2_transform_axes[1].scatter(q2_gray_matter_breakpoints[:,0],q2_gray_matter_breakpo
q2_transform_axes[1].set_title("Gray Matter Intensity Transformation")
q2_transform_axes[1].set_xlabel("Input Intensity")
q2_transform_axes[1].set_ylabel("Output Intensity")
q2_transform_axes[1].set_xlim(0,255)
q2_transform_axes[1].set_ylim(0,255)
q2_transform_axes[1].grid(True)
plt.tight_layout()
plt.show()

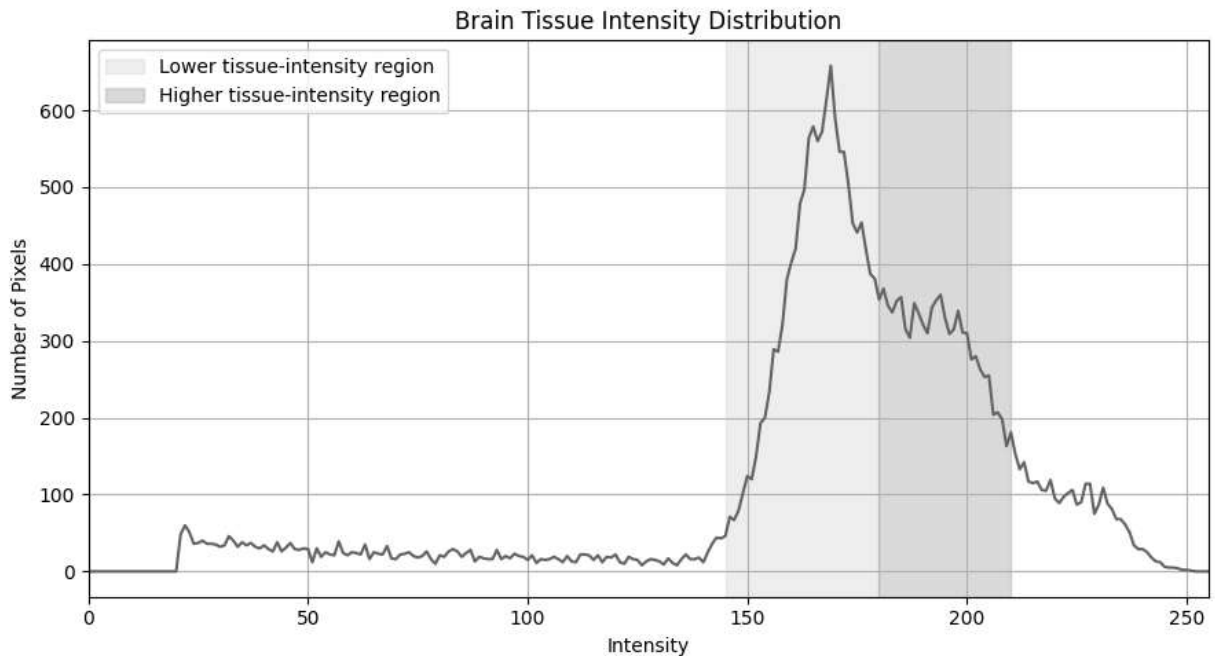
```



```

In [ ]: plt.figure(figsize=(9,5))
plt.plot(np.arange(256),q2_tissue_histogram,linewidth=1.5)
# Lower tissue-intensity region
plt.axvspan(145,180,alpha=0.20,color="orange",label="Lower tissue-intensity region")
# Higher tissue-intensity region
plt.axvspan(180,210,alpha=0.20,color="green",label="Higher tissue-intensity region")
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.title("Brain Tissue Intensity Distribution")
plt.xlim(0,255)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```



Discussion

The histogram shows that most of the relevant brain-tissue intensities are concentrated approximately within the mid-to-high intensity range. Two intensity regions were therefore selected from the histogram to guide the design of the transformations.

Two separate piecewise-linear intensity transformations were designed. The first increases the contrast predominantly within the lower selected tissue-intensity range, with the aim of accentuating white matter. The second places stronger contrast enhancement within the higher selected tissue-intensity range, with the aim of accentuating gray matter. These intensity ranges are used as approximate enhancement regions rather than strict tissue-segmentation boundaries.

The transformations were kept gradual outside the selected ranges in order to preserve the overall anatomical appearance of the proton-density image and avoid excessive saturation caused by stronger contrast mappings.

Question 3 - Gamma Correction

Gamma correction is applied to the lightness (L) channel of the image in the *Lab** color space. Different gamma values are tested and a suitable value is selected based on the resulting image.

The histograms of the original and gamma-corrected images are then compared.

```
In [ ]: q3_image_path = IMAGE_DIR / "q3.jpeg"
        q3_image_bgr = cv.imread(str(q3_image_path))
```

```

if q3_image_bgr is None:
    raise FileNotFoundError(f"Could not load: {q3_image_path}")
q3_image_rgb = cv.cvtColor(q3_image_bgr,cv.COLOR_BGR2RGB)
plt.figure(figsize=(6,6))
plt.imshow(q3_image_rgb)
plt.title("Original Image for Gamma Correction")
plt.axis("off")
plt.show()

```

Original Image for Gamma Correction



```

In [ ]: q3_image_lab = cv.cvtColor(q3_image_bgr,cv.COLOR_BGR2LAB)
q3_lightness_channel,q3_a_channel,q3_b_channel = cv.split(q3_image_lab)
print("L channel shape:",q3_lightness_channel.shape)
print("L channel range:",q3_lightness_channel.min(),"to",q3_lightness_channel.max())

```

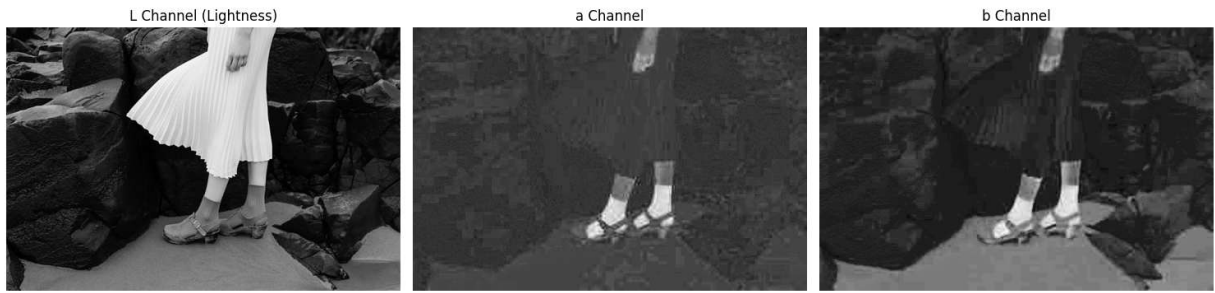
L channel shape: (480, 720)

L channel range: 0 to 255

```

In [ ]: q3_lab_figure,q3_lab_axes = plt.subplots(1,3,figsize=(15,5))
q3_lab_channels = [q3_lightness_channel,q3_a_channel,q3_b_channel]
q3_lab_titles = ["L Channel (Lightness)","a Channel","b Channel"]
for q3_channel_index in range(3):
    q3_lab_axes[q3_channel_index].imshow(q3_lab_channels[q3_channel_index],cmap="gr")
    q3_lab_axes[q3_channel_index].set_title(q3_lab_titles[q3_channel_index])
    q3_lab_axes[q3_channel_index].axis("off")
plt.tight_layout()
plt.show()

```



```
In [ ]: def q3_gamma_correct_lightness(q3_lightness_input,q3_gamma_value):
        q3_normalized_lightness = q3_lightness_input.astype(np.float32)/255.0
        q3_gamma_corrected_normalized = np.power(q3_normalized_lightness,q3_gamma_value)
        q3_gamma_corrected_lightness = np.clip(q3_gamma_corrected_normalized*255.0,0,255)
        return q3_gamma_corrected_lightness.astype(np.uint8)
```

```
In [ ]: q3_gamma_candidates = [0.5,0.7,0.85,1.0]
        q3_gamma_outputs_rgb = []
        for q3_gamma_candidate in q3_gamma_candidates:
            q3_corrected_lightness = q3_gamma_correct_lightness(q3_lightness_channel,q3_gamma_candidate)
            q3_corrected_lab = cv.merge([q3_corrected_lightness,q3_a_channel,q3_b_channel])
            q3_corrected_bgr = cv.cvtColor(q3_corrected_lab,cv.COLOR_LAB2BGR)
            q3_corrected_rgb = cv.cvtColor(q3_corrected_bgr,cv.COLOR_BGR2RGB)
            q3_gamma_outputs_rgb.append(q3_corrected_rgb)
```

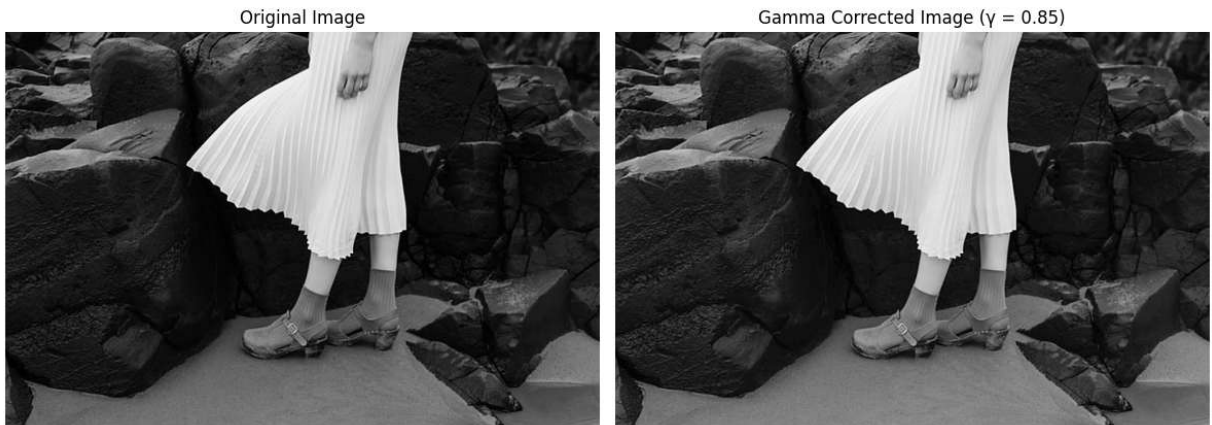
```
In [ ]: q3_gamma_figure,q3_gamma_axes = plt.subplots(1,5,figsize=(22,5))
        # Original
        q3_gamma_axes[0].imshow(q3_image_rgb)
        q3_gamma_axes[0].set_title("Original")
        q3_gamma_axes[0].axis("off")
        # Gamma-corrected versions
        for q3_gamma_index,q3_gamma_candidate in enumerate(q3_gamma_candidates):
            q3_gamma_axes[q3_gamma_index+1].imshow(q3_gamma_outputs_rgb[q3_gamma_index])
            q3_gamma_axes[q3_gamma_index+1].set_title(f"Gamma = {q3_gamma_candidate}")
            q3_gamma_axes[q3_gamma_index+1].axis("off")
        plt.tight_layout()
        plt.show()
```



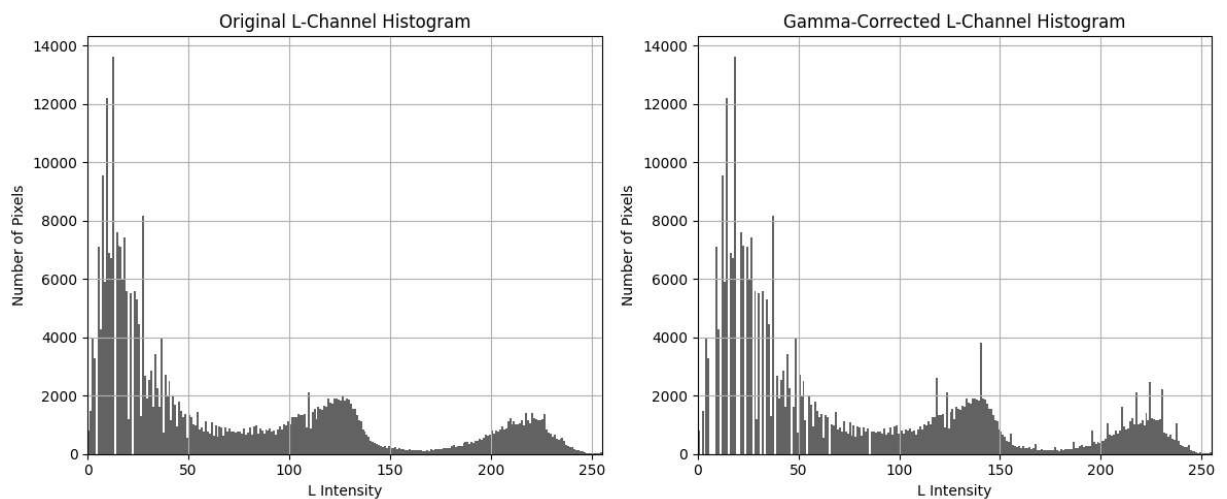
```
In [ ]: q3_selected_gamma = 0.85
        q3_selected_lightness = q3_gamma_correct_lightness(q3_lightness_channel,q3_selected_gamma)
        q3_selected_lab = cv.merge([q3_selected_lightness,q3_a_channel,q3_b_channel])
        q3_selected_bgr = cv.cvtColor(q3_selected_lab,cv.COLOR_LAB2BGR)
        q3_selected_rgb = cv.cvtColor(q3_selected_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q3_final_figure,q3_final_axes = plt.subplots(1,2,figsize=(12,5))
        q3_final_axes[0].imshow(q3_image_rgb)
        q3_final_axes[0].set_title("Original Image")
        q3_final_axes[0].axis("off")
        q3_final_axes[1].imshow(q3_selected_rgb)
```

```
q3_final_axes[1].set_title(f"Gamma Corrected Image ( $\gamma = \{q3\_selected\_gamma\}$ ")
q3_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: q3_histogram_figure, q3_histogram_axes = plt.subplots(1, 2, figsize=(12, 5))
# Original L-channel histogram
q3_histogram_axes[0].hist(q3_lightness_channel.ravel(), bins=256, range=(0, 256))
q3_histogram_axes[0].set_title("Original L-Channel Histogram")
q3_histogram_axes[0].set_xlabel("L Intensity")
q3_histogram_axes[0].set_ylabel("Number of Pixels")
q3_histogram_axes[0].set_xlim(0, 255)
q3_histogram_axes[0].grid(True)
# Gamma-corrected L-channel histogram
q3_histogram_axes[1].hist(q3_selected_lightness.ravel(), bins=256, range=(0, 256))
q3_histogram_axes[1].set_title("Gamma-Corrected L-Channel Histogram")
q3_histogram_axes[1].set_xlabel("L Intensity")
q3_histogram_axes[1].set_ylabel("Number of Pixels")
q3_histogram_axes[1].set_xlim(0, 255)
q3_histogram_axes[1].grid(True)
plt.tight_layout()
plt.show()
```



Discussion

Gamma correction was applied only to the L channel of the Lab* representation so that the image lightness could be adjusted while largely preserving the chromatic information contained in the a and b channels.

Several gamma values were tested. A value of $\gamma = 0.85$ was selected because it moderately brightened the darker regions and revealed additional shadow detail while maintaining a natural overall appearance. Lower gamma values such as 0.5 and 0.7 produced stronger brightening and a more washed-out appearance.

The histogram of the corrected L channel is shifted toward higher intensity values compared with the original histogram. This is consistent with the gamma transformation for $\gamma < 1$, which maps darker and mid-level intensities to brighter output values.

In []:

Question 4 - Vibrance Enhancement

The image is converted to the HSV color space and the saturation channel is modified using the nonlinear transformation specified in the assignment.

Different values of the parameter (a) are tested, and the value producing the most visually pleasing enhancement is selected.

```
In [ ]: q4_image_path = IMAGE_DIR / "q4.jpeg"
q4_image_bgr = cv.imread(str(q4_image_path))
if q4_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 4 image: {q4_image_path}")
q4_image_rgb = cv.cvtColor(q4_image_bgr, cv.COLOR_BGR2RGB)
plt.figure(figsize=(6,6))
plt.imshow(q4_image_rgb)
plt.title("Original Image - Question 4")
plt.axis("off")
plt.show()
```

Original Image - Question 4



```
In [ ]: q4_image_hsv = cv.cvtColor(q4_image_bgr,cv.COLOR_BGR2HSV)
q4_hue_channel,q4_saturation_channel,q4_value_channel = cv.split(q4_image_hsv)
print("Hue range:",q4_hue_channel.min(),"to",q4_hue_channel.max())
print("Saturation range:",q4_saturation_channel.min(),"to",q4_saturation_channel.ma
print("Value range:",q4_value_channel.min(),"to",q4_value_channel.max())
```

Hue range: 0 to 179

Saturation range: 0 to 255

Value range: 0 to 238

```
In [ ]: q4_hsv_figure,q4_hsv_axes = plt.subplots(1,3,figsize=(15,5))
q4_hsv_channels = [q4_hue_channel,q4_saturation_channel,q4_value_channel]
q4_hsv_titles = ["Hue Channel","Saturation Channel","Value Channel"]
for q4_channel_index in range(3):
    q4_hsv_axes[q4_channel_index].imshow(q4_hsv_channels[q4_channel_index],cmap="gr
    q4_hsv_axes[q4_channel_index].set_title(q4_hsv_titles[q4_channel_index])
    q4_hsv_axes[q4_channel_index].axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: def q4_apply_vibrance(q4_saturation_input,q4_a_value,q4_sigma_value=70):
q4_saturation_float = q4_saturation_input.astype(np.float32)
q4_vibrance_boost = q4_a_value*128.0*np.exp(-((q4_saturation_float-128.0)**2)/((
q4_enhanced_saturation = np.minimum(q4_saturation_float+q4_vibrance_boost,255.0
return q4_enhanced_saturation.astype(np.uint8)
```

```
In [ ]:
```

Testing for multiple values of a

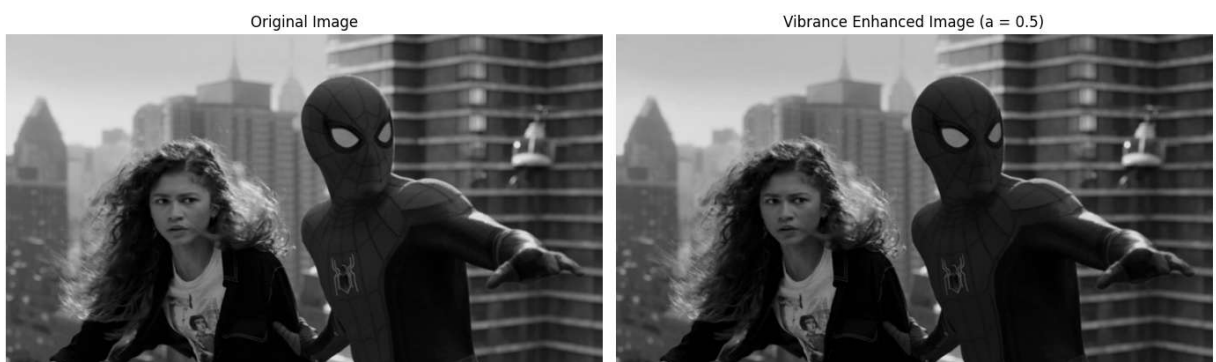
```
In [ ]: q4_a_candidates = [0.25,0.50,0.75,1.00]
q4_vibrance_outputs_rgb = []
for q4_a_candidate in q4_a_candidates:
    q4_corrected_saturation = q4_apply_vibrance(q4_saturation_channel,q4_a_candidate)
    q4_corrected_hsv = cv.merge([q4_hue_channel,q4_corrected_saturation,q4_value_channel])
    q4_corrected_bgr = cv.cvtColor(q4_corrected_hsv,cv.COLOR_HSV2BGR)
    q4_corrected_rgb = cv.cvtColor(q4_corrected_bgr,cv.COLOR_BGR2RGB)
    q4_vibrance_outputs_rgb.append(q4_corrected_rgb)
```

```
In [ ]: q4_compare_figure,q4_compare_axes = plt.subplots(1,5,figsize=(22,5))
q4_compare_axes[0].imshow(q4_image_rgb)
q4_compare_axes[0].set_title("Original")
q4_compare_axes[0].axis("off")
for q4_candidate_index,q4_a_candidate in enumerate(q4_a_candidates):
    q4_compare_axes[q4_candidate_index+1].imshow(q4_vibrance_outputs_rgb[q4_candidate_index])
    q4_compare_axes[q4_candidate_index+1].set_title(f"a = {q4_a_candidate}")
    q4_compare_axes[q4_candidate_index+1].axis("off")
plt.tight_layout()
plt.show()
```

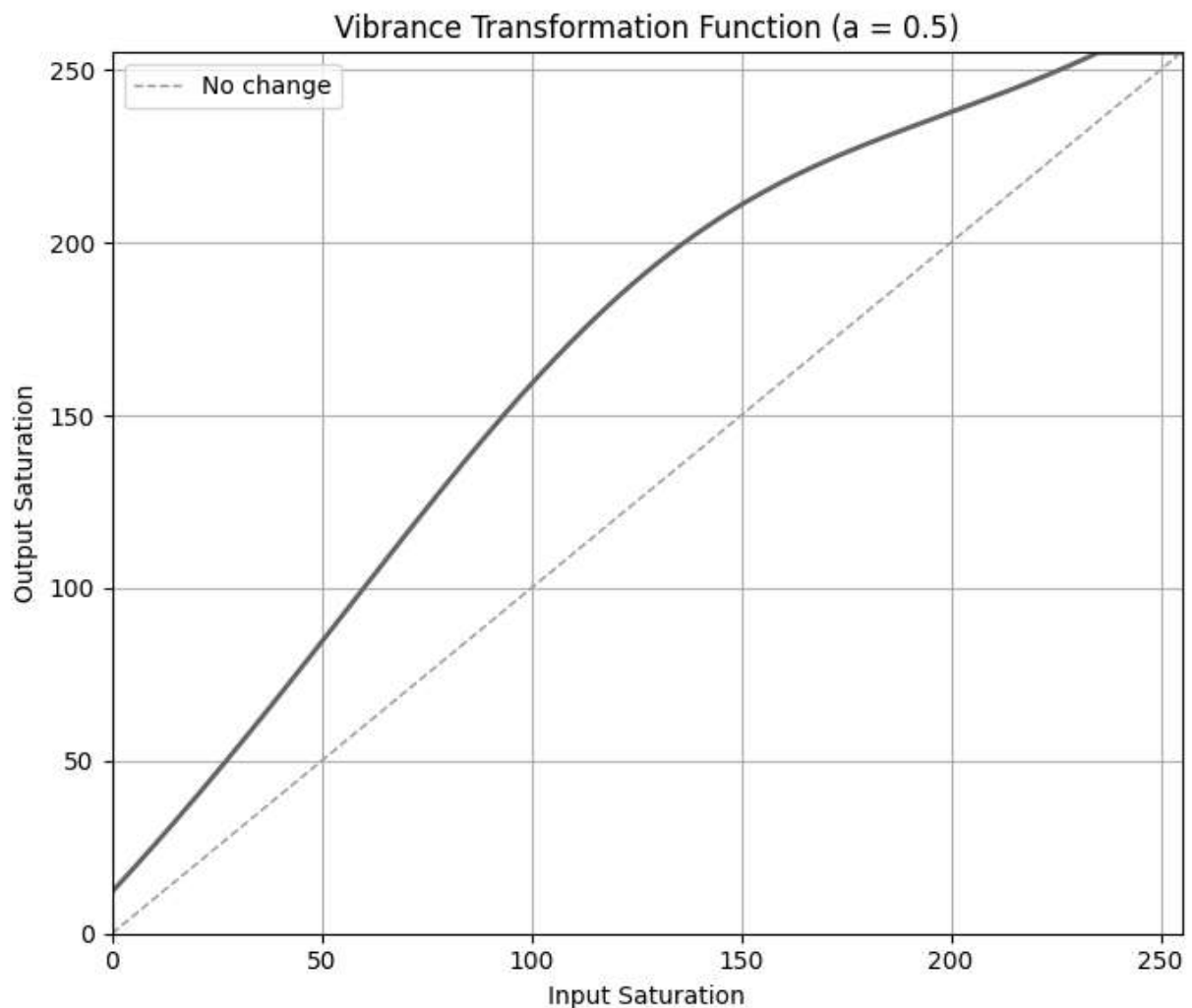


```
In [ ]: q4_selected_a = 0.50
q4_selected_saturation = q4_apply_vibrance(q4_saturation_channel,q4_selected_a)
q4_selected_hsv = cv.merge([q4_hue_channel,q4_selected_saturation,q4_value_channel])
q4_selected_bgr = cv.cvtColor(q4_selected_hsv,cv.COLOR_HSV2BGR)
q4_selected_rgb = cv.cvtColor(q4_selected_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q4_final_figure,q4_final_axes = plt.subplots(1,2,figsize=(14,6))
q4_final_axes[0].imshow(q4_image_rgb)
q4_final_axes[0].set_title("Original Image")
q4_final_axes[0].axis("off")
q4_final_axes[1].imshow(q4_selected_rgb)
q4_final_axes[1].set_title(f"Vibrance Enhanced Image (a = {q4_selected_a})")
q4_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: q4_intensity_axis = np.arange(256,dtype=np.float32)
q4_transformation_curve = np.minimum(q4_intensity_axis+q4_selected_a*128.0*np.exp(-
plt.figure(figsize=(7,6))
plt.plot(q4_intensity_axis,q4_transformation_curve,linewidth=2)
plt.plot([0,255],[0,255],"--",linewidth=1,label="No change")
plt.xlabel("Input Saturation")
plt.ylabel("Output Saturation")
plt.title(f"Vibrance Transformation Function (a = {q4_selected_a})")
plt.xlim(0,255)
plt.ylim(0,255)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Question 5 - Histogram Equalization

Histogram equalization is implemented manually using the image histogram and cumulative distribution function (CDF).

The original and equalized images are compared together with their corresponding histograms.

```
In [ ]: q5_image_path = IMAGE_DIR / "q5.jpeg"
q5_image_gray = cv.imread(str(q5_image_path),cv.IMREAD_GRAYSCALE)
if q5_image_gray is None:
    raise FileNotFoundError(f"Could not load Question 5 image: {q5_image_path}")
print("Image shape:",q5_image_gray.shape)
print("Image datatype:",q5_image_gray.dtype)
print("Minimum intensity:",q5_image_gray.min())
print("Maximum intensity:",q5_image_gray.max())
```

Image shape: (500, 500)

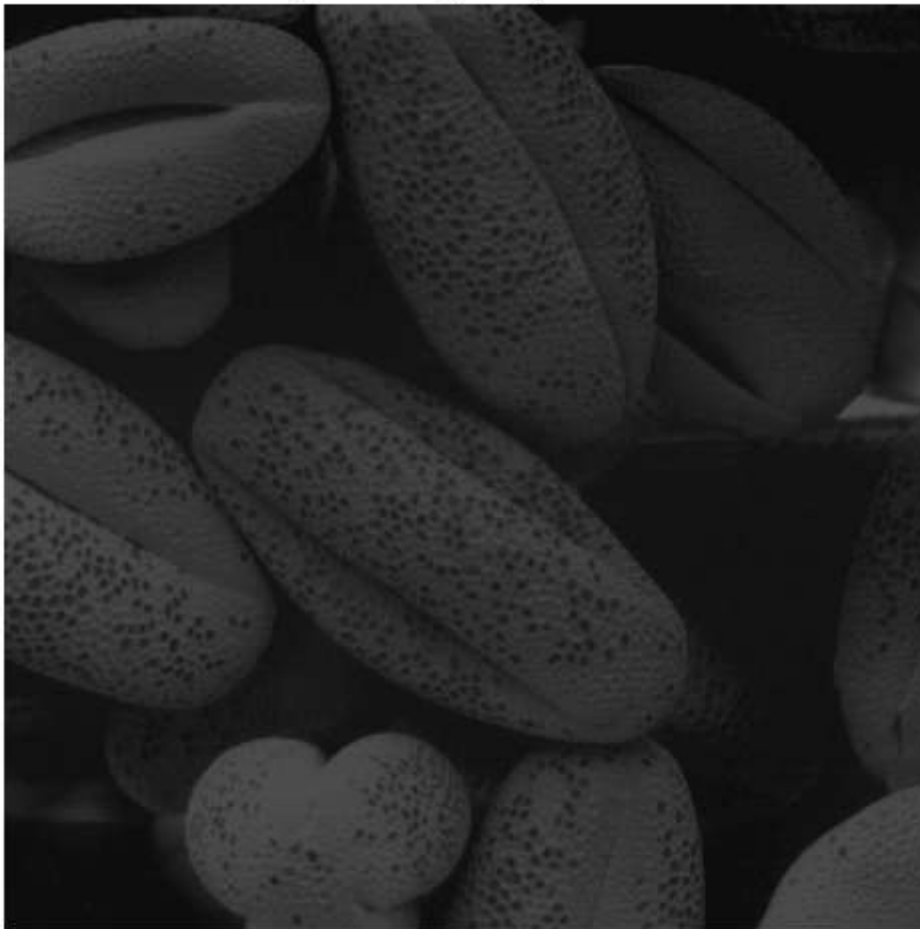
Image datatype: uint8

Minimum intensity: 13

Maximum intensity: 83

```
In [ ]: plt.figure(figsize=(6,6))
plt.imshow(q5_image_gray,cmap="gray",vmin=0,vmax=255)
plt.title("Original Image - Question 5")
plt.axis("off")
plt.show()
```

Original Image - Question 5



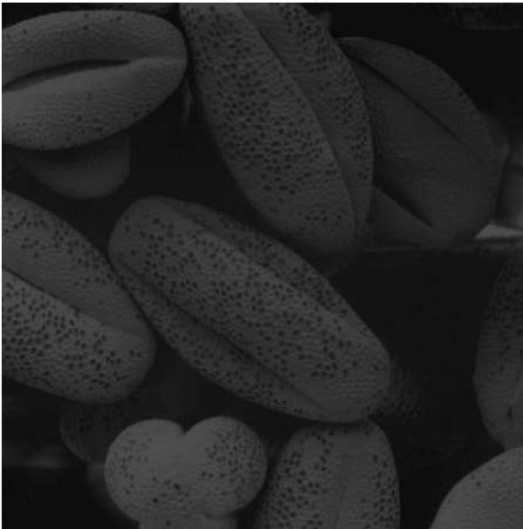
```
In [ ]: def q5_histogram_equalize(q5_input_image):
    q5_histogram = np.bincount(q5_input_image.ravel(),minlength=256)
    q5_probability_distribution = q5_histogram/q5_input_image.size
    q5_cdf = np.cumsum(q5_probability_distribution)
    q5_mapping = np.floor(255*q5_cdf).astype(np.uint8)
```

```
q5_equalized_image = q5_mapping[q5_input_image]
return q5_equalized_image,q5_histogram,q5_cdf,q5_mapping
```

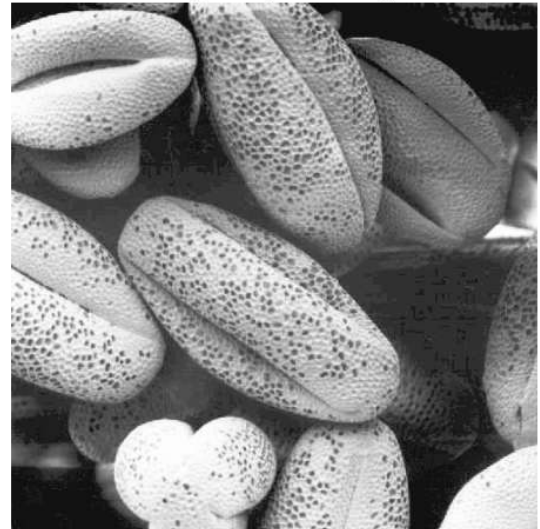
```
In [157... q5_equalized_gray,q5_original_histogram,q5_original_cdf,q5_equalization_mapping = q
```

```
In [ ]: q5_image_figure,q5_image_axes = plt.subplots(1,2,figsize=(12,5))
q5_image_axes[0].imshow(q5_image_gray,cmap="gray",vmin=0,vmax=255)
q5_image_axes[0].set_title("Original Image")
q5_image_axes[0].axis("off")
q5_image_axes[1].imshow(q5_equalized_gray,cmap="gray",vmin=0,vmax=255)
q5_image_axes[1].set_title("Histogram Equalized Image")
q5_image_axes[1].axis("off")
plt.tight_layout()
plt.show()
```

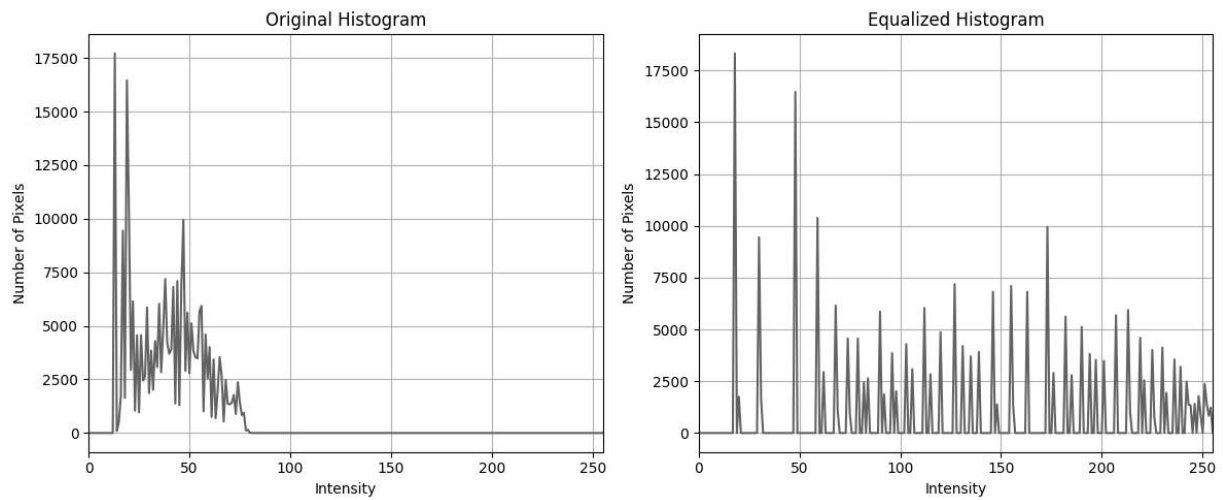
Original Image



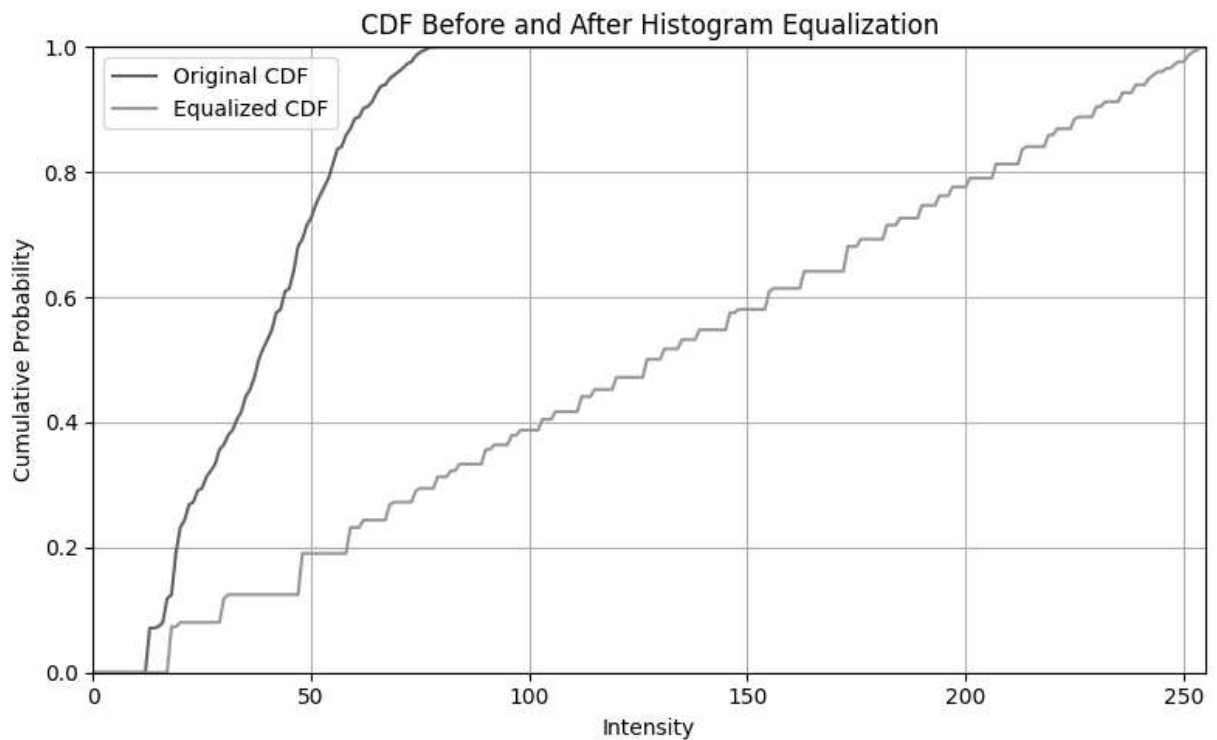
Histogram Equalized Image



```
In [ ]: q5_equalized_histogram = np.bincount(q5_equalized_gray.ravel(),minlength=256)
q5_histogram_figure,q5_histogram_axes = plt.subplots(1,2,figsize=(12,5))
q5_histogram_axes[0].plot(np.arange(256),q5_original_histogram)
q5_histogram_axes[0].set_title("Original Histogram")
q5_histogram_axes[0].set_xlabel("Intensity")
q5_histogram_axes[0].set_ylabel("Number of Pixels")
q5_histogram_axes[0].set_xlim(0,255)
q5_histogram_axes[0].grid(True)
q5_histogram_axes[1].plot(np.arange(256),q5_equalized_histogram)
q5_histogram_axes[1].set_title("Equalized Histogram")
q5_histogram_axes[1].set_xlabel("Intensity")
q5_histogram_axes[1].set_ylabel("Number of Pixels")
q5_histogram_axes[1].set_xlim(0,255)
q5_histogram_axes[1].grid(True)
plt.tight_layout()
plt.show()
```



```
In [ ]: q5_equalized_probability = q5_equalized_histogram/q5_equalized_gray.size
q5_equalized_cdf = np.cumsum(q5_equalized_probability)
plt.figure(figsize=(8,5))
plt.plot(np.arange(256),q5_original_cdf,label="Original CDF")
plt.plot(np.arange(256),q5_equalized_cdf,label="Equalized CDF")
plt.xlabel("Intensity")
plt.ylabel("Cumulative Probability")
plt.title("CDF Before and After Histogram Equalization")
plt.xlim(0,255)
plt.ylim(0,1)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



Discussion

The original image contains most of its pixel intensities within a relatively narrow low-intensity range, which results in poor global contrast.

Histogram equalization redistributes these intensity values over a much wider portion of the available 0–255 range. As a result, previously dark image details become more visible and the overall contrast is improved.

The equalized histogram is not perfectly uniform because the image contains a finite number of discrete intensity values. However, the corresponding CDF is distributed much more gradually across the full intensity range compared with the steep CDF of the original image.

In []:

Question 6 - Histogram Equalization of the Foreground

The image is converted to HSV color space and the hue, saturation, and value planes are examined.

The saturation plane is used to obtain a binary foreground mask because it provides a strong separation between the low-saturation background and the more saturated foreground.

Histogram equalization is then applied only to the value channel within the detected foreground, while the original background is preserved.

```
In [ ]: q6_image_path = IMAGE_DIR / "q6.jpeg"
q6_image_bgr = cv.imread(str(q6_image_path))
if q6_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 6 image: {q6_image_path}")
q6_image_rgb = cv.cvtColor(q6_image_bgr, cv.COLOR_BGR2RGB)
plt.figure(figsize=(6,6))
plt.imshow(q6_image_rgb)
plt.title("Original Image - Question 6")
plt.axis("off")
plt.show()
```

Original Image - Question 6



```
In [ ]: q6_image_hsv = cv.cvtColor(q6_image_bgr,cv.COLOR_BGR2HSV)
        q6_hue_channel,q6_saturation_channel,q6_value_channel = cv.split(q6_image_hsv)
```

```
In [ ]: q6_hsv_figure,q6_hsv_axes = plt.subplots(1,3,figsize=(15,5))
        q6_hsv_channels = [q6_hue_channel,q6_saturation_channel,q6_value_channel]
        q6_hsv_titles = ["Hue Plane","Saturation Plane","Value Plane"]
        for q6_plane_index in range(3):
            q6_hsv_axes[q6_plane_index].imshow(q6_hsv_channels[q6_plane_index],cmap="gray")
            q6_hsv_axes[q6_plane_index].set_title(q6_hsv_titles[q6_plane_index])
            q6_hsv_axes[q6_plane_index].axis("off")
        plt.tight_layout()
        plt.show()
```



```
In [ ]: q6_selected_threshold = 30
        _,q6_foreground_mask = cv.threshold(q6_saturation_channel,q6_selected_threshold,255)
        plt.figure(figsize=(6,5))
        plt.imshow(q6_foreground_mask,cmap="gray",vmin=0,vmax=255)
        plt.title(f"Initial Foreground Mask (T = {q6_selected_threshold})")
        plt.axis("off")
        plt.tight_layout()
        plt.show()
```

Initial Foreground Mask (T = 30)



```
In [ ]: from scipy.ndimage import label
q6_inverse_mask = q6_foreground_mask==0
q6_labels,q6_num_labels = label(q6_inverse_mask)
q6_refined_mask = q6_foreground_mask.copy()
q6_max_hole_size = 3000
for q6_label_index in range(1,q6_num_labels+1):
    q6_region = q6_labels==q6_label_index
    q6_region_size = np.sum(q6_region)
    q6_touches_boundary = np.any(q6_region[0,:]) or np.any(q6_region[-1,:]) or np.a
    if not q6_touches_boundary and q6_region_size<=q6_max_hole_size:
        q6_refined_mask[q6_region] = 255
```

```
In [ ]: q6_foreground_bgr = cv.bitwise_and(q6_image_bgr,q6_image_bgr,mask=q6_refined_mask)
q6_foreground_rgb = cv.cvtColor(q6_foreground_bgr,cv.COLOR_BGR2RGB)
plt.figure(figsize=(6,5))
plt.imshow(q6_foreground_rgb)
plt.title("Extracted Foreground")
plt.axis("off")
plt.tight_layout()
plt.show()
```

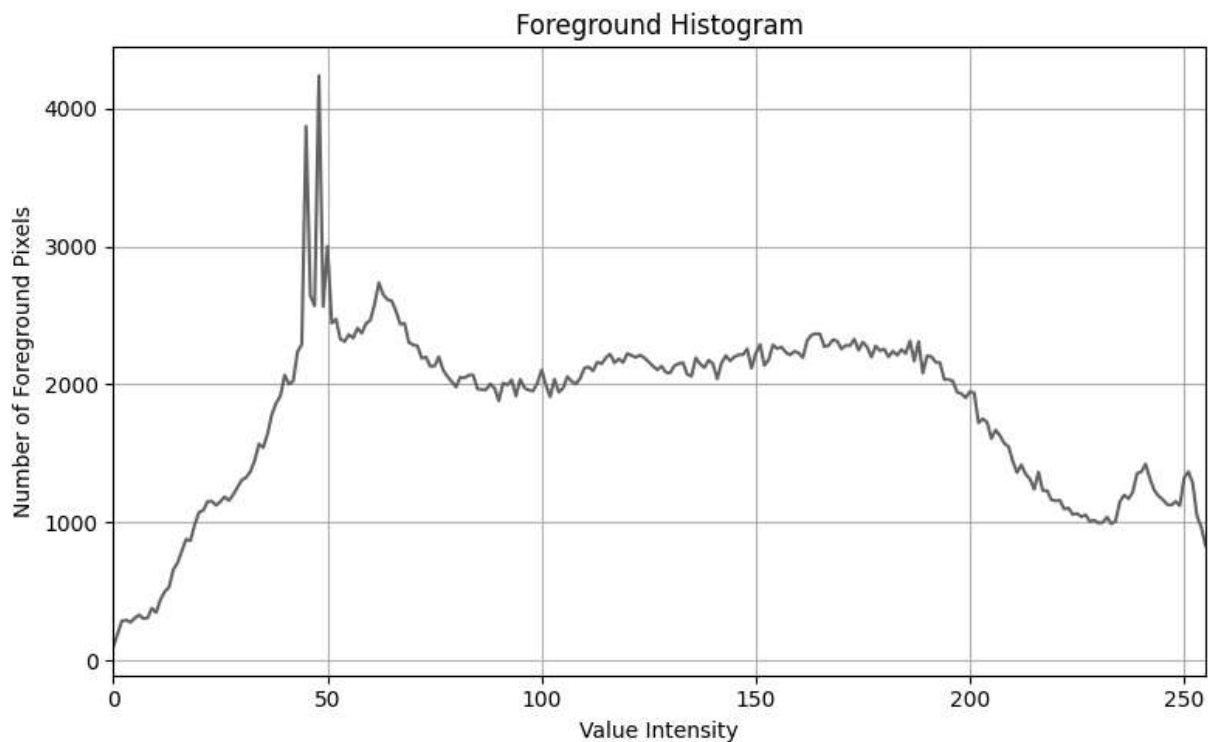
Extracted Foreground



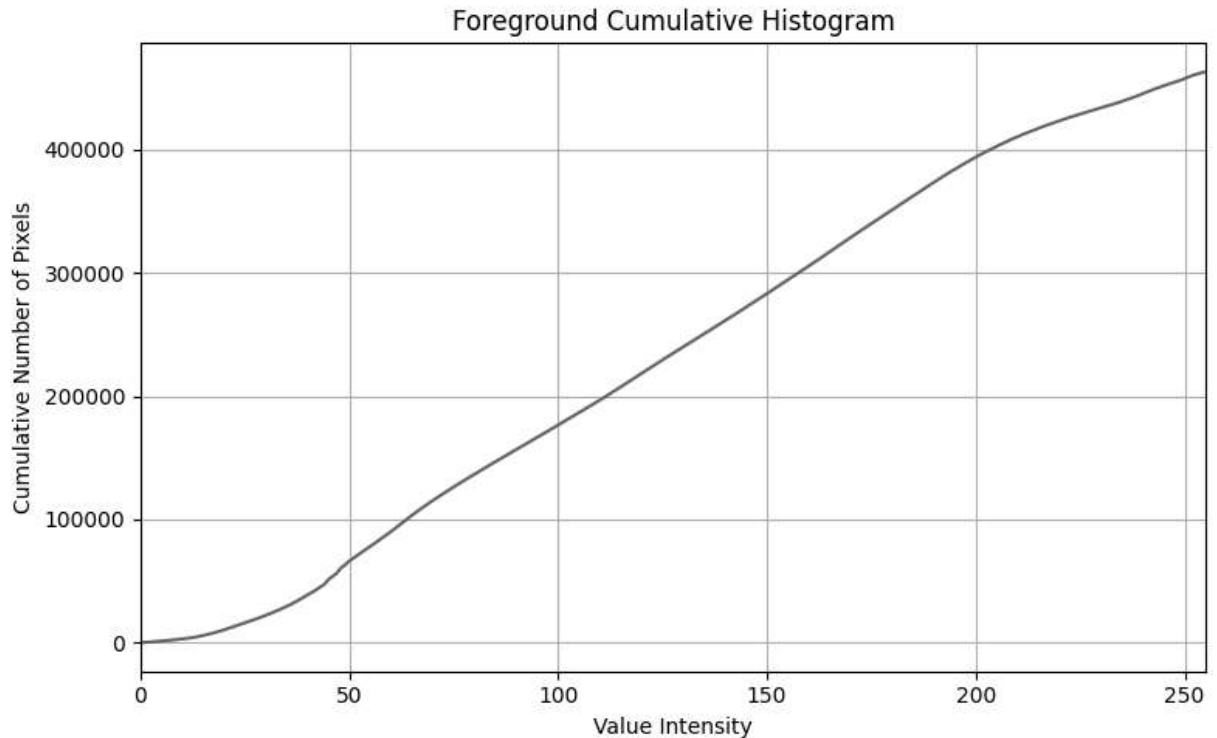
```
In [ ]: q6_foreground_value_pixels = q6_value_channel[q6_refined_mask>0]
        print("Number of foreground pixels:",q6_foreground_value_pixels.size)
```

Number of foreground pixels: 462937

```
In [ ]: q6_foreground_histogram = np.bincount(q6_foreground_value_pixels,minlength=256)
        plt.figure(figsize=(8,5))
        plt.plot(np.arange(256),q6_foreground_histogram)
        plt.xlabel("Value Intensity")
        plt.ylabel("Number of Foreground Pixels")
        plt.title("Foreground Histogram")
        plt.xlim(0,255)
        plt.grid(True)
        plt.tight_layout()
        plt.show()
```



```
In [ ]: q6_foreground_cdf = np.cumsum(q6_foreground_histogram)
plt.figure(figsize=(8,5))
plt.plot(np.arange(256),q6_foreground_cdf)
plt.xlabel("Value Intensity")
plt.ylabel("Cumulative Number of Pixels")
plt.title("Foreground Cumulative Histogram")
plt.xlim(0,255)
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [ ]: q6_nonzero_cdf = q6_foreground_cdf[q6_foreground_histogram>0]
q6_cdf_min = q6_nonzero_cdf[0]
q6_foreground_pixel_count = q6_foreground_value_pixels.size
q6_equalization_mapping = np.round(255*(q6_foreground_cdf-q6_cdf_min)/(q6_foreground_pixel_count-q6_cdf_min))
q6_equalization_mapping = np.clip(q6_equalization_mapping,0,255).astype(np.uint8)

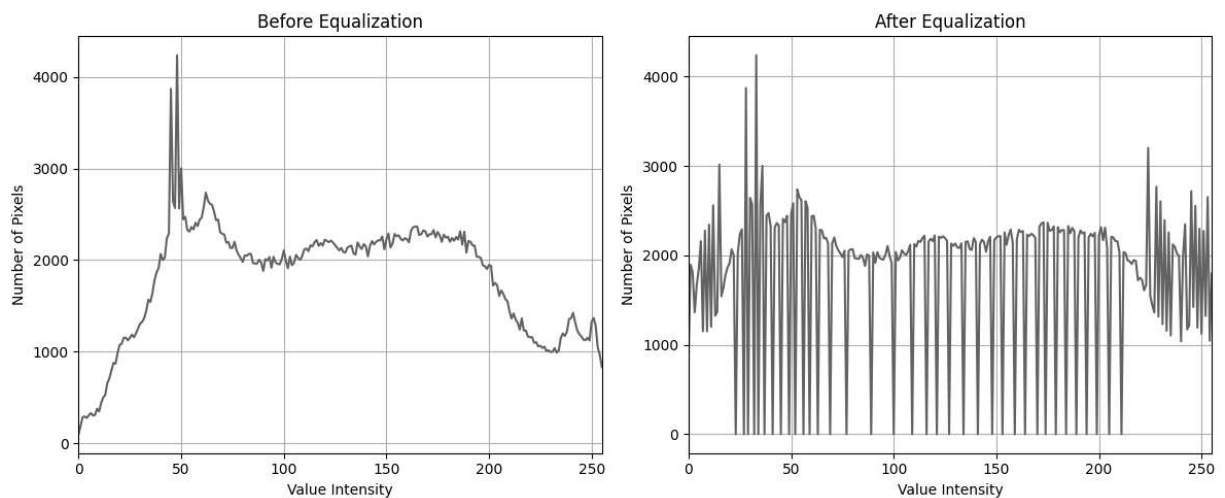
In [ ]: q6_equalized_value_channel = q6_value_channel.copy()
q6_equalized_value_channel[q6_refined_mask>0] = q6_equalization_mapping[q6_value_channel[q6_refined_mask>0]]

In [ ]: q6_equalized_hsv = cv.merge([q6_hue_channel,q6_saturation_channel,q6_equalized_value_channel])
q6_equalized_bgr = cv.cvtColor(q6_equalized_hsv,cv.COLOR_HSV2BGR)
q6_equalized_rgb = cv.cvtColor(q6_equalized_bgr,cv.COLOR_BGR2RGB)

In [ ]: q6_final_figure,q6_final_axes = plt.subplots(1,2,figsize=(12,5))
q6_final_axes[0].imshow(q6_image_rgb)
q6_final_axes[0].set_title("Original Image")
q6_final_axes[0].axis("off")
q6_final_axes[1].imshow(q6_equalized_rgb)
q6_final_axes[1].set_title("Foreground Histogram Equalized")
q6_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: q6_equalized_foreground_pixels = q6_equalized_value_channel[q6_refined_mask>0]
q6_equalized_foreground_histogram = np.bincount(q6_equalized_foreground_pixels,min1
q6_hist_figure,q6_hist_axes = plt.subplots(1,2,figsize=(12,5))
q6_hist_axes[0].plot(np.arange(256),q6_foreground_histogram)
q6_hist_axes[0].set_title("Before Equalization")
q6_hist_axes[0].set_xlabel("Value Intensity")
q6_hist_axes[0].set_ylabel("Number of Pixels")
q6_hist_axes[0].set_xlim(0,255)
q6_hist_axes[0].grid(True)
q6_hist_axes[1].plot(np.arange(256),q6_equalized_foreground_histogram)
q6_hist_axes[1].set_title("After Equalization")
q6_hist_axes[1].set_xlabel("Value Intensity")
q6_hist_axes[1].set_ylabel("Number of Pixels")
q6_hist_axes[1].set_xlim(0,255)
q6_hist_axes[1].grid(True)
plt.tight_layout()
plt.show()
```



Discussion

Histogram equalization redistributes the foreground intensity values over a wider range, increasing the contrast of the subject. The before-and-after histograms show this redistribution of intensity values. Since the transformation is applied only within the foreground mask, the background is preserved while the contrast and visible details of the foreground are enhanced.

In []:

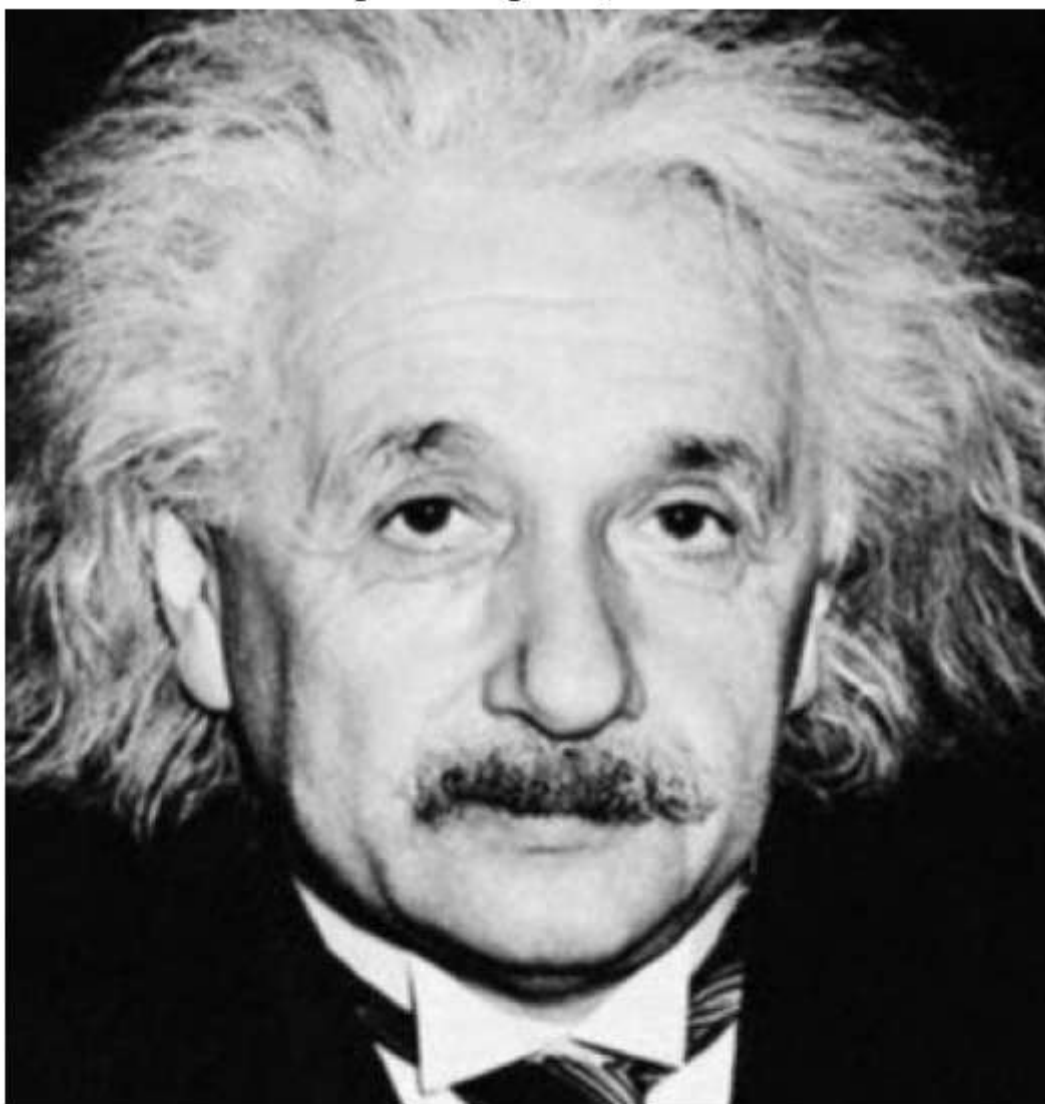
Question 7 – Sobel Filtering

The Sobel operator is used to estimate image intensity gradients and thereby detect edges. In this section, Sobel filtering is performed using three approaches: OpenCV's `filter2D`, a manually implemented convolution, and separable 1-D filtering using the separability of the Sobel kernel. The resulting edge maps are compared.

```
In [ ]: q7_image_path = IMAGE_DIR / "q7.jpeg"
q7_image_gray = cv.imread(str(q7_image_path), cv.IMREAD_GRAYSCALE)

if q7_image_gray is None:
    raise FileNotFoundError(f"Could not load Question 7 image: {q7_image_path}")
plt.figure(figsize=(6, 6))
plt.imshow(q7_image_gray, cmap="gray")
plt.title("Original Image - Question 7")
plt.axis("off")
plt.tight_layout()
plt.show()
```

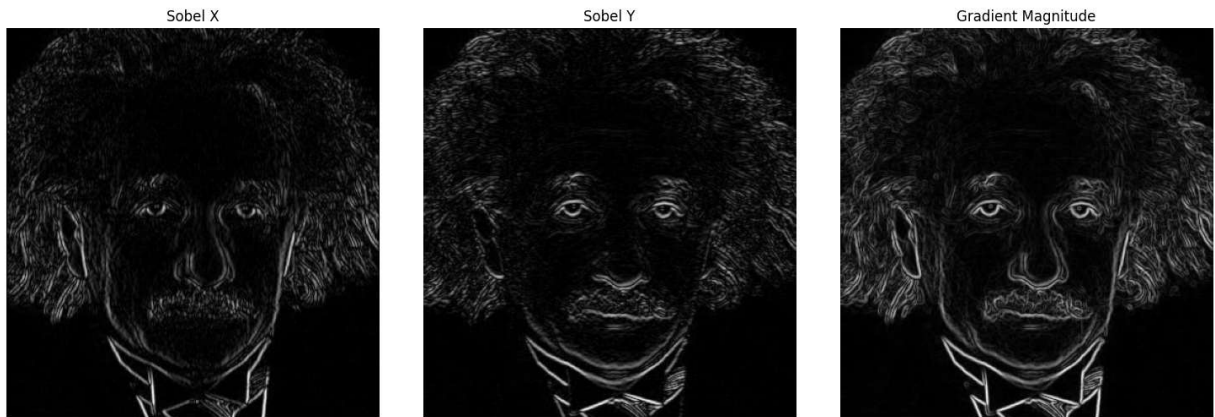
Original Image - Question 7



```
In [ ]: q7_sobel_x_kernel = np.array([[1,0,-1],[2,0,-2],[1,0,-1]],dtype=np.float32)
        q7_sobel_y_kernel = np.array([[1,2,1],[0,0,0],[-1,-2,-1]],dtype=np.float32)
```

```
In [ ]: q7_gradient_x_filter2d = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_
        q7_gradient_y_filter2d = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_
        q7_gradient_magnitude_filter2d = np.sqrt(q7_gradient_x_filter2d**2+q7_gradient_y_fi
        q7_gradient_magnitude_filter2d = cv.normalize(q7_gradient_magnitude_filter2d,None,0
```

```
In [ ]: q7_filter2d_figure,q7_filter2d_axes = plt.subplots(1,3,figsize=(15,5))
        q7_filter2d_axes[0].imshow(np.abs(q7_gradient_x_filter2d),cmap="gray")
        q7_filter2d_axes[0].set_title("Sobel X")
        q7_filter2d_axes[1].imshow(np.abs(q7_gradient_y_filter2d),cmap="gray")
        q7_filter2d_axes[1].set_title("Sobel Y")
        q7_filter2d_axes[2].imshow(q7_gradient_magnitude_filter2d,cmap="gray")
        q7_filter2d_axes[2].set_title("Gradient Magnitude")
        for q7_axis in q7_filter2d_axes:
            q7_axis.axis("off")
        plt.tight_layout()
        plt.show()
```



```
In [ ]: def q7_manual_filter(q7_input_image,q7_kernel):
        q7_kernel_height,q7_kernel_width = q7_kernel.shape
        q7_pad_y = q7_kernel_height//2
        q7_pad_x = q7_kernel_width//2
        q7_padded_image = np.pad(q7_input_image.astype(np.float32),((q7_pad_y,q7_pad_y)
        q7_output_image = np.zeros_like(q7_input_image,dtype=np.float32)
        for q7_row in range(q7_input_image.shape[0]):
            for q7_col in range(q7_input_image.shape[1]):
                q7_local_region = q7_padded_image[q7_row:q7_row+q7_kernel_height,q7_col:q7_col+q7_kernel_width]
                q7_output_image[q7_row,q7_col] = np.sum(q7_local_region*q7_kernel)
        return q7_output_image
```

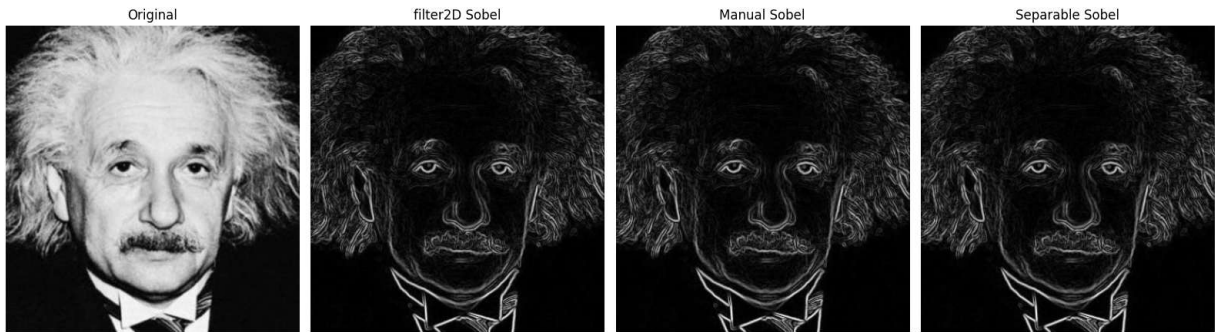
```
In [ ]: q7_gradient_x_manual = q7_manual_filter(q7_image_gray,q7_sobel_x_kernel)
        q7_gradient_y_manual = q7_manual_filter(q7_image_gray,q7_sobel_y_kernel)
        q7_gradient_magnitude_manual = np.sqrt(q7_gradient_x_manual**2+q7_gradient_y_manual**2)
        q7_gradient_magnitude_manual = cv.normalize(q7_gradient_magnitude_manual,None,0,255)
```

```
In [ ]: q7_smoothing_column = np.array([[1],[2],[1]],dtype=np.float32)
        q7_derivative_row = np.array([[1,0,-1]],dtype=np.float32)
        q7_derivative_column = np.array([[1],[0],[-1]],dtype=np.float32)
        q7_smoothing_row = np.array([[1,2,1]],dtype=np.float32)
```

```
In [ ]: q7_temp_x = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_smoothing_col)
        q7_gradient_x_separable = cv.filter2D(q7_temp_x,cv.CV_32F,q7_derivative_row)
        q7_temp_y = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_smoothing_row)
        q7_gradient_y_separable = cv.filter2D(q7_temp_y,cv.CV_32F,q7_derivative_column)
        q7_gradient_magnitude_separable = np.sqrt(q7_gradient_x_separable**2+q7_gradient_y_separable**2)
        q7_gradient_magnitude_separable = cv.normalize(q7_gradient_magnitude_separable,None,0,255)
```

```
In [ ]: q7_compare_figure,q7_compare_axes = plt.subplots(1,4,figsize=(16,5))
        q7_compare_axes[0].imshow(q7_image_gray,cmap="gray")
        q7_compare_axes[0].set_title("Original")
        q7_compare_axes[1].imshow(q7_gradient_magnitude_filter2d,cmap="gray")
        q7_compare_axes[1].set_title("filter2D Sobel")
        q7_compare_axes[2].imshow(q7_gradient_magnitude_manual,cmap="gray")
        q7_compare_axes[2].set_title("Manual Sobel")
        q7_compare_axes[3].imshow(q7_gradient_magnitude_separable,cmap="gray")
        q7_compare_axes[3].set_title("Separable Sobel")
        for q7_axis in q7_compare_axes:
            q7_axis.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Discussion

The Sobel operator estimates image gradients in the horizontal and vertical directions, and the gradient magnitude highlights strong intensity transitions corresponding to edges.

The OpenCV `filter2D`, manually implemented convolution, and separable Sobel implementations produce visually similar edge maps. The separable implementation obtains the same Sobel operation using consecutive one-dimensional filters, reducing the computational cost compared with direct two-dimensional convolution.

In []:

Question 8 – Image Zooming Using Interpolation

Image zooming is implemented for a scale factor $s \in (0, 10]$ using nearest-neighbor and bilinear interpolation. The provided downscaled images are enlarged by a factor of 4 and compared with their corresponding original images using the normalized sum of squared difference (SSD).

```
In [ ]: q8_original_1_bgr = cv.imread(str(IMAGE_DIR/"im01.png"))
q8_small_1_bgr = cv.imread(str(IMAGE_DIR/"im01small.png"))
q8_original_2_bgr = cv.imread(str(IMAGE_DIR/"im02.png"))
q8_small_2_bgr = cv.imread(str(IMAGE_DIR/"im02small.png"))
if any(q8_image is None for q8_image in [q8_original_1_bgr, q8_small_1_bgr, q8_original_2_bgr, q8_small_2_bgr]):
    raise FileNotFoundError("Could not load one or more Question 8 images.")
q8_original_1_rgb = cv.cvtColor(q8_original_1_bgr, cv.COLOR_BGR2RGB)
q8_small_1_rgb = cv.cvtColor(q8_small_1_bgr, cv.COLOR_BGR2RGB)
q8_original_2_rgb = cv.cvtColor(q8_original_2_bgr, cv.COLOR_BGR2RGB)
q8_small_2_rgb = cv.cvtColor(q8_small_2_bgr, cv.COLOR_BGR2RGB)
```

```
In [ ]: print("Image 1 Original:", q8_original_1_rgb.shape)
print("Image 1 Small:   ", q8_small_1_rgb.shape)
print("Image 2 Original:", q8_original_2_rgb.shape)
print("Image 2 Small:   ", q8_small_2_rgb.shape)
```

Image 1 Original: (1080, 1920, 3)
 Image 1 Small: (270, 480, 3)
 Image 2 Original: (1200, 1920, 3)
 Image 2 Small: (300, 480, 3)

```
In [ ]: q8_input_figure,q8_input_axes = plt.subplots(2,2,figsize=(10,8))
q8_input_axes[0,0].imshow(q8_original_1_rgb)
q8_input_axes[0,0].set_title("Image 1 - Original")
q8_input_axes[0,1].imshow(q8_small_1_rgb)
q8_input_axes[0,1].set_title("Image 1 - Small")
q8_input_axes[1,0].imshow(q8_original_2_rgb)
q8_input_axes[1,0].set_title("Image 2 - Original")
q8_input_axes[1,1].imshow(q8_small_2_rgb)
q8_input_axes[1,1].set_title("Image 2 - Small")
for q8_axis in q8_input_axes.ravel():
    q8_axis.axis("off")
plt.tight_layout()
plt.show()
```

Image 1 - Original



Image 1 - Small



Image 2 - Original



Image 2 - Small



```
In [ ]: def q8_zoom_nearest(q8_input_image,q8_scale_factor):
q8_input_height,q8_input_width = q8_input_image.shape[:2]
q8_output_height = int(round(q8_input_height*q8_scale_factor))
q8_output_width = int(round(q8_input_width*q8_scale_factor))
q8_output_image = np.zeros((q8_output_height,q8_output_width,q8_input_image.sha
for q8_output_row in range(q8_output_height):
    for q8_output_col in range(q8_output_width):
        q8_source_row = int(round(q8_output_row/q8_scale_factor))
        q8_source_col = int(round(q8_output_col/q8_scale_factor))
        q8_source_row = min(q8_source_row,q8_input_height-1)
        q8_source_col = min(q8_source_col,q8_input_width-1)
```

```

        q8_output_image[q8_output_row,q8_output_col] = q8_input_image[q8_source
return q8_output_image

```

```

In [ ]: def q8_zoom_bilinear(q8_input_image,q8_scale_factor):
        q8_input_height,q8_input_width = q8_input_image.shape[:2]
        q8_output_height = int(round(q8_input_height*q8_scale_factor))
        q8_output_width = int(round(q8_input_width*q8_scale_factor))
        q8_output_image = np.zeros((q8_output_height,q8_output_width,q8_input_image.sha
        for q8_output_row in range(q8_output_height):
            for q8_output_col in range(q8_output_width):
                q8_source_y = q8_output_row/q8_scale_factor
                q8_source_x = q8_output_col/q8_scale_factor
                q8_y0 = int(np.floor(q8_source_y))
                q8_x0 = int(np.floor(q8_source_x))
                q8_y1 = min(q8_y0+1,q8_input_height-1)
                q8_x1 = min(q8_x0+1,q8_input_width-1)
                q8_y0 = min(q8_y0,q8_input_height-1)
                q8_x0 = min(q8_x0,q8_input_width-1)
                q8_dy = q8_source_y-q8_y0
                q8_dx = q8_source_x-q8_x0
                q8_top = (1-q8_dx)*q8_input_image[q8_y0,q8_x0]+q8_dx*q8_input_image[q8_
                q8_bottom = (1-q8_dx)*q8_input_image[q8_y1,q8_x0]+q8_dx*q8_input_image[
                q8_output_image[q8_output_row,q8_output_col] = (1-q8_dy)*q8_top+q8_dy*q
        return np.clip(q8_output_image,0,255).astype(np.uint8)

```

```

In [ ]: q8_scale_factor = 4
        q8_image_1_nearest = q8_zoom_nearest(q8_small_1_rgb,q8_scale_factor)
        q8_image_1_bilinear = q8_zoom_bilinear(q8_small_1_rgb,q8_scale_factor)
        q8_image_2_nearest = q8_zoom_nearest(q8_small_2_rgb,q8_scale_factor)
        q8_image_2_bilinear = q8_zoom_bilinear(q8_small_2_rgb,q8_scale_factor)

```

```

In [ ]: q8_compare_1_figure,q8_compare_1_axes = plt.subplots(1,3,figsize=(15,5))
        q8_compare_1_axes[0].imshow(q8_original_1_rgb)
        q8_compare_1_axes[0].set_title("Original Image 1")
        q8_compare_1_axes[1].imshow(q8_image_1_nearest)
        q8_compare_1_axes[1].set_title("Nearest Neighbor x4")
        q8_compare_1_axes[2].imshow(q8_image_1_bilinear)
        q8_compare_1_axes[2].set_title("Bilinear x4")
        for q8_axis in q8_compare_1_axes:
            q8_axis.axis("off")
        plt.tight_layout()
        plt.show()

```



```

In [ ]: q8_compare_2_figure,q8_compare_2_axes = plt.subplots(1,3,figsize=(15,5))
        q8_compare_2_axes[0].imshow(q8_original_2_rgb)
        q8_compare_2_axes[0].set_title("Original Image 2")
        q8_compare_2_axes[1].imshow(q8_image_2_nearest)

```

```

q8_compare_2_axes[1].set_title("Nearest Neighbor x4")
q8_compare_2_axes[2].imshow(q8_image_2_bilinear)
q8_compare_2_axes[2].set_title("Bilinear x4")
for q8_axis in q8_compare_2_axes:
    q8_axis.axis("off")
plt.tight_layout()
plt.show()

```



```

In [ ]: def q8_normalized_ssd(q8_reference_image,q8_test_image):
        q8_reference_float = q8_reference_image.astype(np.float32)
        q8_test_float = q8_test_image.astype(np.float32)
        q8_squared_difference = (q8_reference_float-q8_test_float)**2
        q8_normalized_ssd_value = np.mean(q8_squared_difference)/(255.0**2)
        return q8_normalized_ssd_value

```

```

In [ ]: print("Image 1 original:",q8_original_1_rgb.shape)
        print("Image 1 nearest:",q8_image_1_nearest.shape)
        print("Image 1 bilinear:",q8_image_1_bilinear.shape)
        print()
        print("Image 2 original:",q8_original_2_rgb.shape)
        print("Image 2 nearest:",q8_image_2_nearest.shape)
        print("Image 2 bilinear:",q8_image_2_bilinear.shape)

```

Image 1 original: (1080, 1920, 3)

Image 1 nearest: (1080, 1920, 3)

Image 1 bilinear: (1080, 1920, 3)

Image 2 original: (1200, 1920, 3)

Image 2 nearest: (1200, 1920, 3)

Image 2 bilinear: (1200, 1920, 3)

```

In [ ]: q8_ssd_1_nearest = q8_normalized_ssd(q8_original_1_rgb,q8_image_1_nearest)
        q8_ssd_1_bilinear = q8_normalized_ssd(q8_original_1_rgb,q8_image_1_bilinear)
        q8_ssd_2_nearest = q8_normalized_ssd(q8_original_2_rgb,q8_image_2_nearest)
        q8_ssd_2_bilinear = q8_normalized_ssd(q8_original_2_rgb,q8_image_2_bilinear)
        print("Image 1")
        print(f"Nearest-neighbor normalized SSD: {q8_ssd_1_nearest:.6f}")
        print(f"Bilinear normalized SSD: {q8_ssd_1_bilinear:.6f}")
        print()
        print("Image 2")
        print(f"Nearest-neighbor normalized SSD: {q8_ssd_2_nearest:.6f}")
        print(f"Bilinear normalized SSD: {q8_ssd_2_bilinear:.6f}")

```

Image 1

Nearest-neighbor normalized SSD: 0.003926

Bilinear normalized SSD: 0.003080

Image 2

Nearest-neighbor normalized SSD: 0.000994

Bilinear normalized SSD: 0.000753

Discussion

Both interpolation methods successfully enlarged the downscaled images by a factor of 4. Nearest-neighbor interpolation copies the value of the closest source pixel and therefore tends to produce more visible block-like artifacts. Bilinear interpolation estimates each new pixel from neighboring source pixels, producing smoother results.

The normalized SSD values provide a quantitative comparison with the supplied original images. A lower normalized SSD indicates that the reconstructed image is closer to the corresponding original.

In []:

Question 9 – Foreground Segmentation and Background Blurring

GrabCut is used to separate the foreground flower from the background. The segmented foreground and background are then combined after applying strong Gaussian blurring to the background to produce a shallow-depth-of-field effect.

In []:

```
q9_image_path = IMAGE_DIR/"q9.jpeg"
q9_image_bgr = cv.imread(str(q9_image_path))
if q9_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 9 image: {q9_image_path}")
q9_image_rgb = cv.cvtColor(q9_image_bgr, cv.COLOR_BGR2RGB)
print("Image shape:", q9_image_bgr.shape)
plt.figure(figsize=(6,6))
plt.imshow(q9_image_rgb)
plt.title("Original Image - Question 9")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Image shape: (841, 561, 3)

Original Image - Question 9



```
In [ ]: q9_height,q9_width = q9_image_bgr.shape[:2]
q9_grabcut_mask = np.zeros((q9_height,q9_width),dtype=np.uint8)
q9_background_model = np.zeros((1,65),dtype=np.float64)
q9_foreground_model = np.zeros((1,65),dtype=np.float64)
q9_rectangle = (int(0.05*q9_width),int(0.05*q9_height),int(0.90*q9_width),int(0.90*
cv.grabCut(q9_image_bgr,q9_grabcut_mask,q9_rectangle,q9_background_model,q9_foregro
```

```

Out[ ]: (array([[0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               ...,
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0]], dtype=uint8),
array([[3.53544480e-01, 5.73278971e-02, 3.33214543e-01, 1.89751691e-01,
        6.61613880e-02, 3.76531315e+01, 5.86275587e+01, 4.21065218e+01,
        1.23411258e+02, 1.33148899e+02, 1.14685609e+02, 5.32975040e+01,
        9.96523984e+01, 6.79825850e+01, 6.87298512e+01, 1.33361051e+02,
        1.03647570e+02, 4.42652897e+01, 8.42831453e+01, 7.42036220e+01,
        5.69561944e+01, 1.18483362e+02, 7.75757964e+01, 1.18483362e+02,
        2.90933895e+02, 1.83298307e+02, 7.75757964e+01, 1.83298307e+02,
        1.21875925e+02, 6.36157543e+02, 4.43328877e+02, 4.68668405e+02,
        4.43328877e+02, 3.48109400e+02, 3.67373831e+02, 4.68668405e+02,
        3.67373831e+02, 4.01494746e+02, 2.23701939e+01, 3.86798605e+01,
        3.02364439e+01, 3.86798605e+01, 1.31674799e+02, 8.09869990e+01,
        3.02364439e+01, 8.09869990e+01, 7.57627917e+01, 8.92748946e+01,
        7.52197463e+01, 1.00216894e+02, 7.52197463e+01, 1.54419419e+02,
        1.60592212e+02, 1.00216894e+02, 1.60592212e+02, 2.01687866e+02,
        8.63005921e+02, 1.13452952e+03, 1.15328138e+03, 1.13452952e+03,
        1.98737093e+03, 2.07729365e+03, 1.15328138e+03, 2.07729365e+03,
        2.31537380e+03]]),
array([[ 0.00000000e+00,  7.71142684e-01,  1.64948454e-02,
         2.11842675e-01,  5.19795547e-04,  3.55000000e+00,
         3.02875000e+01,  2.67750000e+01,  9.39636345e+01,
         2.16921472e+02,  2.37909553e+02,  5.20740546e+01,
         1.10159664e+02,  1.15988971e+02,  3.66921441e+01,
         1.75415082e+02,  2.16543982e+02,  5.47500000e+01,
         9.73833333e+01,  8.64833333e+01,  4.49975000e+01,
         5.88668750e+01,  4.58487500e+01,  5.88668750e+01,
         1.62254844e+02,  1.06952188e+02,  4.58487500e+01,
         1.06952188e+02,  1.15499375e+02,  1.32822969e+03,
         2.87341397e+02,  1.40692160e+02,  2.87341397e+02,
         1.12628550e+02,  7.94295639e+01,  1.40692160e+02,
         7.94295639e+01,  7.22110569e+01,  7.32387386e+01,
         1.03578512e+02,  5.21310689e+01,  1.03578512e+02,
         3.50371566e+02,  2.59353652e+02,  5.21310689e+01,
         2.59353652e+02,  3.54535067e+02,  7.68856192e+02,
         1.45421246e+02, -2.90101211e+02,  1.45421246e+02,
         5.98706699e+02,  4.58700510e+02, -2.90101211e+02,
         4.58700510e+02,  7.71078557e+02,  6.72208333e+01,
         5.74458333e+01,  3.01875000e+01,  5.74458333e+01,
         6.18030556e+01,  3.67313889e+01,  3.01875000e+01,
         3.67313889e+01,  2.81497222e+01]]))

```

```

In [ ]: q9_binary_mask = np.where((q9_grabcut_mask==cv.GC_FGD)|(q9_grabcut_mask==cv.GC_PR_F

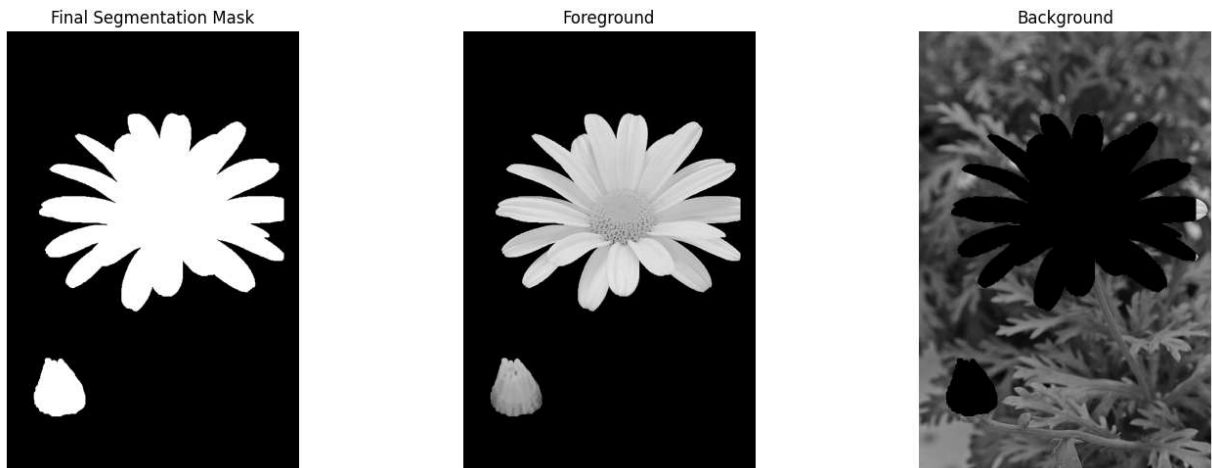
```

```

In [ ]: q9_foreground_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_binary_mask)
q9_background_mask = cv.bitwise_not(q9_binary_mask)
q9_background_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_background_mas
q9_foreground_rgb = cv.cvtColor(q9_foreground_bgr,cv.COLOR_BGR2RGB)
q9_background_rgb = cv.cvtColor(q9_background_bgr,cv.COLOR_BGR2RGB)

```

```
In [ ]: q9_segment_figure,q9_segment_axes = plt.subplots(1,3,figsize=(15,5))
q9_segment_axes[0].imshow(q9_binary_mask,cmap="gray")
q9_segment_axes[0].set_title("Final Segmentation Mask")
q9_segment_axes[1].imshow(q9_foreground_rgb)
q9_segment_axes[1].set_title("Foreground")
q9_segment_axes[2].imshow(q9_background_rgb)
q9_segment_axes[2].set_title("Background")
for q9_axis in q9_segment_axes:
    q9_axis.axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: # Remove small false foreground regions by keeping only the largest connected compo
q9_num_labels,q9_labels,q9_stats,q9_centroids = cv.connectedComponentsWithStats(q9_
# Ignore Label 0 because it is the background
q9_largest_label = 1+np.argmax(q9_stats[1:,cv.CC_STAT_AREA])
q9_final_mask = np.where(q9_labels==q9_largest_label,255,0).astype(np.uint8)
# Display comparison
q9_mask_compare_fig,q9_mask_compare_axes = plt.subplots(1,2,figsize=(10,5))
q9_mask_compare_axes[0].imshow(q9_binary_mask,cmap="gray")
q9_mask_compare_axes[0].set_title("Original GrabCut Mask")
q9_mask_compare_axes[1].imshow(q9_final_mask,cmap="gray")
q9_mask_compare_axes[1].set_title("Final Cleaned Mask")
for ax in q9_mask_compare_axes:
    ax.axis("off")
plt.tight_layout()
plt.show()
```

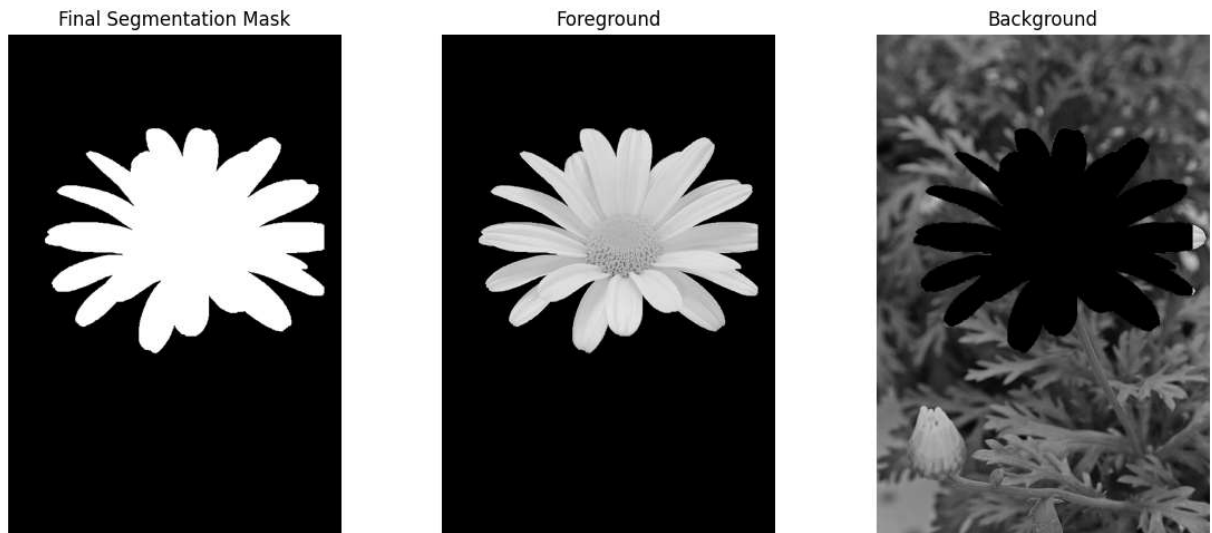
Original GrabCut Mask



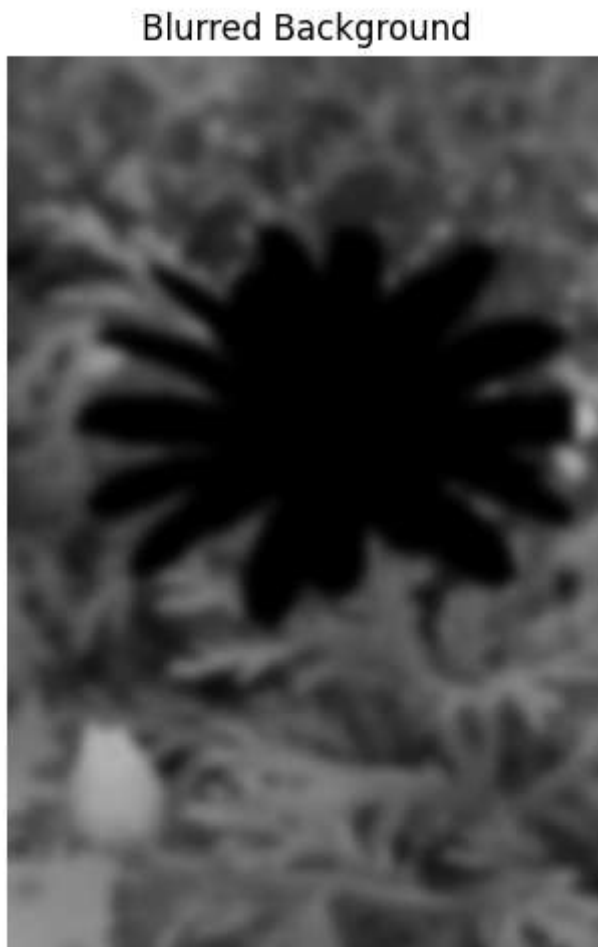
Final Cleaned Mask



```
In [ ]: # Extract foreground using cleaned mask
q9_foreground_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_final_mask)
# Extract background
q9_background_mask = cv.bitwise_not(q9_final_mask)
q9_background_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_background_mas
# Convert for Matplotlib
q9_foreground_rgb = cv.cvtColor(q9_foreground_bgr,cv.COLOR_BGR2RGB)
q9_background_rgb = cv.cvtColor(q9_background_bgr,cv.COLOR_BGR2RGB)
# Display final segmentation results
fig,axes = plt.subplots(1,3,figsize=(12,5))
axes[0].imshow(q9_final_mask,cmap="gray")
axes[0].set_title("Final Segmentation Mask")
axes[1].imshow(q9_foreground_rgb)
axes[1].set_title("Foreground")
axes[2].imshow(q9_background_rgb)
axes[2].set_title("Background")
for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: q9_blurred_background_bgr = cv.GaussianBlur(q9_background_bgr,(41,41),0)
q9_blurred_background_rgb = cv.cvtColor(q9_blurred_background_bgr,cv.COLOR_BGR2RGB)
plt.figure(figsize=(6,5))
plt.imshow(q9_blurred_background_rgb)
plt.title("Blurred Background")
plt.axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: q9_enhanced_bgr = cv.add(q9_foreground_bgr,q9_blurred_background_bgr)
q9_enhanced_rgb = cv.cvtColor(q9_enhanced_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q9_final_figure,q9_final_axes = plt.subplots(1,2,figsize=(12,5))
q9_final_axes[0].imshow(q9_image_rgb)
q9_final_axes[0].set_title("Original Image")
q9_final_axes[0].axis("off")
q9_final_axes[1].imshow(q9_enhanced_rgb)
q9_final_axes[1].set_title("Enhanced Image")
q9_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```

Original Image



Enhanced Image



Discussion

GrabCut was used to segment the flower from the background. The background was then strongly blurred while the segmented flower was kept sharp and recombined with the blurred background.

The region just outside the flower can appear darker because the extracted background contains black pixels where the flower was removed. During Gaussian blurring, these zero-valued pixels are averaged with nearby background pixels, reducing their intensity close to the flower boundary.

```
In [ ]:
```

Question 10 – Bilateral Filtering

Bilateral filtering is used to smooth the Lake Minaret image while preserving sharp edges. The result is compared with Gaussian smoothing using a similar spatial kernel. The bilateral filter is implemented using both OpenCV and a custom implementation, and the two bilateral-filter outputs are compared quantitatively.

```
In [ ]: q10_image_path = IMAGE_DIR/"q10.jpeg"
q10_image_bgr = cv.imread(str(q10_image_path))
if q10_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 10 image: {q10_image_path}")
q10_image_rgb = cv.cvtColor(q10_image_bgr, cv.COLOR_BGR2RGB)
plt.figure(figsize=(7,5))
plt.imshow(q10_image_rgb)
plt.title("Original Lake Minaret Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Original Lake Minaret Image



```
In [ ]: q10_sigma_s = 3.0
q10_sigma_r = 50.0
q10_kernel_size = 9
q10_radius = q10_kernel_size // 2
```

```
In [ ]: q10_gaussian_bgr = cv.GaussianBlur(q10_image_bgr, (q10_kernel_size, q10_kernel_size),
q10_opencv_bilateral_bgr = cv.bilateralFilter(q10_image_bgr, d=q10_kernel_size, sigma
```

```
q10_gaussian_rgb = cv.cvtColor(q10_gaussian_bgr,cv.COLOR_BGR2RGB)
q10_opencv_bilateral_rgb = cv.cvtColor(q10_opencv_bilateral_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q10_part_a_figure,q10_part_a_axes = plt.subplots(1,3,figsize=(15,5))
q10_part_a_axes[0].imshow(q10_image_rgb)
q10_part_a_axes[0].set_title("Original")
q10_part_a_axes[1].imshow(q10_gaussian_rgb)
q10_part_a_axes[1].set_title("Gaussian Blur")
q10_part_a_axes[2].imshow(q10_opencv_bilateral_rgb)
q10_part_a_axes[2].set_title("OpenCV Bilateral Filter")
for q10_axis in q10_part_a_axes:
    q10_axis.axis("off")
plt.tight_layout()
plt.show()
```



```
In [ ]: def q10_custom_bilateral_filter(q10_input_image,q10_sigma_space,q10_sigma_range,q10
q10_input_float = q10_input_image.astype(np.float32)
q10_height,q10_width = q10_input_float.shape[:2]
q10_padded_image = cv.copyMakeBorder(q10_input_float,q10_filter_radius,q10_filt
q10_coordinate_range = np.arange(-q10_filter_radius,q10_filter_radius+1)
q10_grid_x,q10_grid_y = np.meshgrid(q10_coordinate_range,q10_coordinate_range)
q10_spatial_weights = np.exp(-(q10_grid_x**2+q10_grid_y**2)/(2.0*q10_sigma_spac
q10_output_image = np.zeros_like(q10_input_float)
for q10_row in range(q10_height):
    for q10_col in range(q10_width):
        q10_patch = q10_padded_image[q10_row:q10_row+2*q10_filter_radius+1,q10_
q10_center_pixel = q10_padded_image[q10_row+q10_filter_radius,q10_col+q
q10_color_difference_squared = np.sum((q10_patch-q10_center_pixel)**2,a
q10_range_weights = np.exp(-q10_color_difference_squared/(2.0*q10_sigma
q10_total_weights = q10_spatial_weights*q10_range_weights
q10_weight_sum = np.sum(q10_total_weights)
q10_output_image[q10_row,q10_col] = np.sum(q10_patch*q10_total_weights[
return np.clip(q10_output_image,0,255).astype(np.uint8)
```

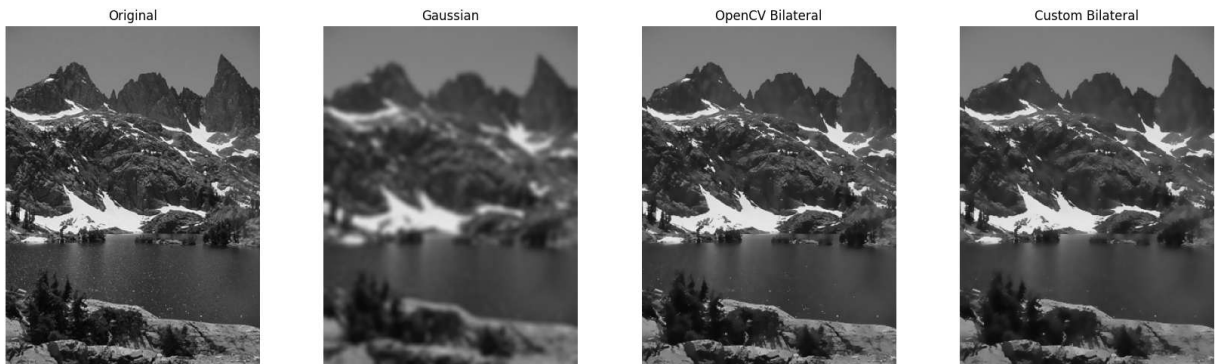
```
In [ ]: q10_custom_bilateral_bgr = q10_custom_bilateral_filter(q10_image_bgr,q10_sigma_s,q1
q10_custom_bilateral_rgb = cv.cvtColor(q10_custom_bilateral_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q10_final_figure,q10_final_axes = plt.subplots(1,4,figsize=(17,5))
q10_final_images = [q10_image_rgb,q10_gaussian_rgb,q10_opencv_bilateral_rgb,q10_cus
q10_final_titles = ["Original","Gaussian","OpenCV Bilateral","Custom Bilateral"]
for q10_index in range(4):
```

```

q10_final_axes[q10_index].imshow(q10_final_images[q10_index])
q10_final_axes[q10_index].set_title(q10_final_titles[q10_index])
q10_final_axes[q10_index].axis("off")
plt.tight_layout()
plt.show()

```



```

In [ ]: q10_opencv_float = q10_opencv_bilateral_bgr.astype(np.float64)
q10_custom_float = q10_custom_bilateral_bgr.astype(np.float64)
q10_mse = np.mean((q10_opencv_float-q10_custom_float)**2)
if q10_mse==0:
    q10_psnr = float("inf")
else:
    q10_psnr = 10*np.log10((255.0**2)/q10_mse)
print(f"MSE between OpenCV and custom bilateral: {q10_mse:.4f}")
print(f"PSNR between OpenCV and custom bilateral: {q10_psnr:.2f} dB")

```

MSE between OpenCV and custom bilateral: 15.5296

PSNR between OpenCV and custom bilateral: 36.22 dB

Discussion

Gaussian filtering smooths the image but also blurs strong edges because the weights depend only on spatial distance. Bilateral filtering additionally considers the difference between neighboring pixel values, allowing smoothing within similar regions while reducing averaging across strong edges.

The OpenCV and custom bilateral-filter results should therefore appear similar, although small differences can occur due to implementation details. The MSE and PSNR provide a quantitative measure of the agreement between the two bilateral-filter implementations.

In []:

In []:

In []:

In []:

In []: